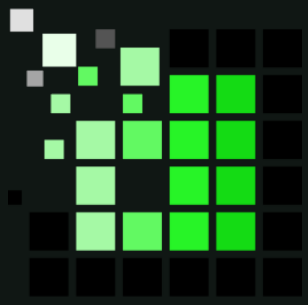


DISPLAYWX

Guia da biblioteca

Interfaces gráficas para ESP32

Telas, toque, widgets e câmera MIPI.
Com exemplos comentados e guia de DMA.

Biblioteca 1.0.30 • Edição 04/11/2026

Manual navegável
Use o sumário e os marcadores do leitor de PDF.
Clique no rodapé para voltar ao índice.

Sumário clicável

Selecione um capítulo para ir diretamente ao conteúdo.

Guia da biblioteca DisplayWX 4

1. O que esta biblioteca faz . . . 4
2. Os dois arquivos do projeto . . . 4
3. Peças ligadas na placa . . . 5
4. Telas suportadas . . . 6
5. Toques suportados . . . 12
6. Câmeras suportadas . . . 14
7. Placas prontas . . . 16
8. Exemplos de combinações . . . 17
9. Rotação, cores, IPS, espelho e SPI compartilhado . . . 26
10. Sketch base dos widgets . . . 29
11. Funções que todo widget tem . . . 31
12. Label . . . 32
13. Led . . . 33
14. CircleButton . . . 33
15. RectButton . . . 34
16. TextButton . . . 35
17. ToggleButton . . . 36
18. CheckBox . . . 37
19. RadioGroup . . . 38
20. TouchArea . . . 39
21. HSlider . . . 39

22. SpinBox . . . 40

23. NumberBox . . . 41

24. TextBox . . . 42

25. VBar . . . 43

26. VAnalog . . . 44

27. CircularBar . . . 45

28. Gauge . . . 46

29. Thermometer . . . 47

30. LineChart . . . 48

31. Image . . . 48

32. CameraMipi . . . 50

Como mudar um exemplo com segurança . . . 52

Guia do DMA da câmera **53**

1. O que é DMA . . . 53

2. Por que a câmera precisa disso . . . 53

3. O caminho da foto . . . 53

4. O que o exemplo pede . . . 54

5. Quem repete isso sozinho . . . 55

6. A foto grande vira o retângulo da tela . . . 56

7. A cópia que o processador faz . . . 56

Guia da biblioteca DisplayWX

Versão da biblioteca: **1.0.29**.

Este texto ensina a usar a biblioteca para desenhar em telas do ESP32. A linguagem é simples. Cada linha de código tem um comentário. O comentário diz o que a linha faz e o que você pode mudar.

1. O que esta biblioteca faz

A DisplayWX desenha botões, textos, gráficos e a imagem da câmera numa tela ligada ao ESP32.

A tela pode usar estes fios:

• **SPI**: poucos fios. Serve para telas pequenas e médias.

• **I2C**: dois fios, SDA e SCL. Serve para telas pequenas, como OLED.

• **RGB**: muitos fios. Serve para placas ESP32-S3 com a tela já soldada.

• **DSI**: cabo MIPI. Serve para telas grandes no ESP32-P4.

O toque pode ir no mesmo SPI da tela, ou num I2C separado.

A câmera MIPI só funciona no **ESP32-P4**, com o pacote de placa na versão que usa ESP-IDF 5.4 ou mais novo.

Os exemplos de widgets deste guia usam sempre a mesma placa:

• Placa: **ESP32-P4**

• Tela: **Guition 7 polegadas, chip JD9165, 1024 por 600**

• Toque: **GT911**

• Câmera: **OV5647**

2. Os dois arquivos do projeto

2.1 O arquivo user_setup.h

Este arquivo fica na pasta da biblioteca, ao lado da pasta `src`. A IDE Arduino lê ele em todo sketch que usa a DisplayWX.

Ele escolhe a família da tela, o chip de toque e o chip da câmera. Coloque **um** define de tela, **um** define de toque e, se houver câmera, **um** define de câmera.

Tela colorida (SPI, RGB, DSI ou 8 bits):

```
#define DISP_DEFAULT // Liga a família de telas coloridas.
```

Tela Nokia 5110:

```
#define DISP_PCD8544 // Liga a tela PCD8544. O gerador às vezes escreve PDC8544. Na biblioteca o nome
```

```
    certo é PCD8544.
```

Tela OLED SSD1306:

```
#define DISP_SSD1306 // Liga a tela SSD1306. O gerador às vezes escreve SD1306. Na biblioteca o nome
    certo é SSD1306.
```

Tela ST7565:

```
#define DISP_U8G2 // Liga a tela monocromática ST7565 pela biblioteca U8g2.
```

Se nenhum desses quatro existir, a compilação para com o aviso para escolher um modelo em `user_setup.h`.

Toque. Use só uma destas linhas:

```
#define TOUCH_GT911            // Toque capacitivo GT911, fios I2C.
#define TOUCH_GT911_WAVESHARE  // GT911 do painel Waveshare de 7 polegadas.
#define TOUCH_XPT2046          // Toque resistivo. Usa os fios SPI da tela.
#define TOUCH_FT6236U          // Toque capacitivo FT6236.
#define TOUCH_FT6336           // Toque capacitivo FT6336. Escreva TOUCH_FT6336. O nome TOUCH_FT6336U
    não liga este driver.
#define TOUCH_CST816           // Toque capacitivo CST816.
#define TOUCH_CST820           // Toque capacitivo CST820.
#define TOUCH_GSL3680          // Toque capacitivo GSL3680, usado no Waveshare de 10 polegadas.
```

Câmera. Use só uma destas linhas, e só no ESP32-P4:

```
#define CAM_OV5647   // Sensor OV5647. Dá para escolher o tamanho da imagem.
#define CAM_OV02C10  // Sensor OV02C10.
#define CAM_SC2336   // Sensor SC2336.
#define CAM_OV2710   // Sensor OV2710. Usado na placa ESP32-P4-EYE.
```

Cores, se a imagem sair errada. As duas linhas são opcionais. Veja a seção 9.

```
#define IS_BGR             // Troca o vermelho com o azul.
#define INVERT_COLORS_BITS  // Inverte claro e escuro.
```

Nas telas PCD8544, SSD1306 e ST7565 a biblioteca desliga a maior parte dos widgets. Sobram texto, LED e barra em algumas delas. A ST7565 também desliga texto, LED, termômetro e barra. Para botões e gráficos, use uma tela colorida (`DISP_DEFAULT`).

2.2 O sketch .ino

O sketch cria a tela, liga o toque, cria os widgets e chama `myDisplay.createTask(false, 3)`. Essa tarefa lê o toque e desenha. O `loop` serve para ler botão, potenciômetro e mudar o valor dos widgets.

Mude o valor do widget no `loop`. A tarefa da biblioteca desenha a tela.

3. Peças ligadas na placa

Os exemplos que falam com o mundo usam quatro peças. Troque o número do pino se a sua placa já usa esse GPIO. Na base Guition 7, os pinos 7, 8, 22, 23 e 27 já pertencem à tela, ao toque e à luz de fundo.

Peça	Pino do exemplo	Ligação
Botão	4	Pulldown. Solto = 0. Apertado = 1.
LED	5	Saída digital. 1 acende. 0 apaga.

Peça	Pino do exemplo	Ligação
Potenciômetro	6	Entrada analógica. O pino precisa ser ADC.
LED com brilho	3	Saída PWM. O número muda o brilho.

Botão em pulldown:

```
3V3 ---- botão ---- GPIO 4
                     |
                  resistor 10k
                     |
                    GND
```

LED:

```
GPIO 5 ---- resistor 330 ---- LED ---- GND
```

Potenciômetro:

```
3V3 ---- um lado do potenciômetro
GPIO 6 ---- o pino do meio
GND ---- o outro lado
```

`analogRead` costuma devolver de 0 a 4095. Se a barra não chegar no topo, troque 4095 pelo maior número que aparecer na serial.

O LED de brilho usa o mesmo desenho do LED, no GPIO 3.

4. Telas suportadas

A coluna **Define** é a linha que entra em `user_setup.h`. A coluna **Fios** diz como a tela fala com o chip.

Nome na biblioteca	Define	Fios	Pinos que o gerador pede
ILI9488	`DISP_DEFAULT`	SPI	CS, MISO, MOSI, DC, CLK, RST
GC9A01	`DISP_DEFAULT`	SPI	CS, MISO, MOSI, DC, CLK, RST
ST7735	`DISP_DEFAULT`	SPI	CS, MISO, MOSI, DC, CLK, RST
ST7789 SPI	`DISP_DEFAULT`	SPI	CS, MISO, MOSI, DC, CLK, RST
ST7796	`DISP_DEFAULT`	SPI	CS, MISO, MOSI, DC, CLK, RST
ILI9225	`DISP_DEFAULT`	SPI	CS, MISO, MOSI, DC, CLK, RST
ST7789 8 bits	`DISP_DEFAULT`	barramento de 8 bits	pinos fixos da placa
ST7789 soldada na Guition 2.8	`DISP_DEFAULT`	SPI da placa	pinos fixos da placa
ST7796 soldada	`DISP_DEFAULT`	SPI da placa	pinos fixos da placa
ST7789 SPI DMA	`DISP_DEFAULT`	SPI DMA	usada na ESP32-P4-EYE
ESP32-8048S043	`DISP_DEFAULT`	RGB	pinos fixos da placa S3

Nome na biblioteca	Define	Fios	Pinos que o gerador pede
ESP32-4827S043	`DISP_DEFAULT`	RGB	pinos fixos da placa S3
ESP32-8048S070	`DISP_DEFAULT`	RGB	pinos fixos da placa S3
ST7262 Waveshare 4.3	`DISP_DEFAULT`	RGB	pinos fixos da placa
JD9365 DSI	`DISP_DEFAULT`	DSI	cabo MIPI, reset da tela
JD9165 DSI	`DISP_DEFAULT`	DSI	cabo MIPI, reset da tela
ST7701 DSI	`DISP_DEFAULT`	DSI	cabo MIPI, reset da tela
ILI9881C DSI	`DISP_DEFAULT`	DSI	cabo MIPI e chip auxiliar Waveshare
EK79007 DSI	`DISP_DEFAULT`	DSI	cabo MIPI, reset da tela
PCD8544	`DISP_PCD8544`	SPI	CS, RST, DC, CLK, DIN
SSD1306	`DISP_SSD1306`	I2C	SDA, SCL, RST, endereço
ST7565	`DISP_U8G2`	SPI	DT, CS, DC, RST

Cores prontas para as telas coloridas: `CFK_WHITE`, `CFK_BLACK`, `CFK_RED`, `CFK_GREEN`, `CFK_BLUE`, `CFK_YELLOW`, `CFK_ORANGE`, `CFK_GRAY`, `CFK_LIME`. Elas já passam pelo ajuste de BGR e de inversão.

O fundo da tela usa `myDisplay.setBackgroundColor(CFK_WHITE)`. O valor entre parênteses é um número de 16 bits. Use um nome `CFK_`. Essa chamada pinta a tela inteira. Ela também guarda a cor. Os widgets usam essa cor para apagar o texto antigo. Chame a função depois de `setDrawObject`. Se chamar antes, a cor fica guardada e a tela ainda não é pintada. Troque `CFK_WHITE` pela cor que você quiser. O detalhe está na seção 9.8.

4.1 Como iniciar cada tela

Esta seção é a receita de cada painel do arquivo `boards.json`. Os números saíram dos exemplos da biblioteca. Troque um GPIO só se a sua placa usar outro fio. Não invente tempo de DSI nem tabela de início.

Ordem do sketch:

• Um define de tela em `user_setup.h`. Só um.

• O bloco do painel, nesta seção.

• Se houver toque, o bloco da seção 5.1, depois de `setDrawObject`.

• Se houver câmera, o bloco da seção 6.1, antes de criar o painel.

• `loadWidgets();` depois `myDisplay.loadScreen(screen0);` depois `myDisplay.createTask(false, 3);`.

• Dentro de `screen0`, `myDisplay.setBackgroundColor` e `myDisplay.drawWidgetsOnScreen(0);`.

Telas SPI coloridas

Vale para ILI9488, GC9A01, ST7735, ST7789 SPI, ST7789 soldada na Guition 2.8, ILI9225, ST7796 e ST7796 soldada. O define é `DISP_DEFAULT`. O barramento é o mesmo. Muda a classe do `new`.

Os GPIO abaixo são os do exemplo ST7789 de 240 por 240. Na Guition 2.8 os fios são outros. Eles estão no bloco seguinte.

```
#if defined(CONFIG_IDF_TARGET_ESP32S3)
SPIClass spi_shared(FSPI); // No ESP32-S3 o SPI da tela usa FSPI.
#else
SPIClass spi_shared(HSPI); // No ESP32 clássico o SPI da tela usa HSPI.
#endif
spi_shared.begin(18, -1, 23); // CLK 18, MISO -1, MOSI 23. Troque se o fio for outro.
bus = new Arduino_HWSPI(2, 5, 18, 23, -1, &spi_shared); // DC 2, CS 5, CLK, MOSI, MISO.
tft = new Arduino_ST7789(bus, 4, rotationScreen, false); // RST 4. Troque a classe pela tabela.
tft->begin(27000000); // Velocidade do SPI. Se a imagem falhar, use 20000000.
myDisplay.setDrawObject(tft); // Entrega a tela. Sem isso o fundo não pinta.
```

Nome no catálogo	Classe no lugar de `Arduino_ST7789`
ILI9488	`Arduino_ILI9488_18bit(bus, DISP_RST, rotationScreen, isIPS)`
GC9A01	`Arduino_GC9A01(bus, DISP_RST, rotationScreen, isIPS)`
ST7735	`Arduino_ST7735(bus, DISP_RST, rotationScreen, isIPS)`
ST7789 - SPI	`Arduino_ST7789(bus, DISP_RST, rotationScreen, isIPS)`
ST7789 - EMBED	`Arduino_ST7789(bus, DISP_RST, rotationScreen, isIPS)`
ILI9225	`Arduino_ILI9225(bus, DISP_RST)`
ST7796	`Arduino_ST7796(bus, DISP_RST, rotationScreen, isIPS, 320, 480)`
ST7796 - EMBED	`Arduino_ST7796(bus, DISP_RST, rotationScreen, isIPS, 320, 480)`

O ILI9225 não recebe rotação nem IPS nesse construtor. O ST7796 de 3,5 pol. recebe largura 320 e altura 480. No GC9A01 sem fio MISO, o exemplo usa `GFX_NOT_DEFINED` no lugar do MISO.

Guition 2.8, exemplo `Arduino1x__ESP32EMBED__GUITION_28`. Tamanho 320 por 240. MOSI 13, MISO 12, CLK 14, CS 15, DC 2, RST -1. A luz de fundo é o GPIO 27 em HIGH. O toque é CST820. Veja a seção 5.1.

ST7789 de 8 bits

Placa ESP32-S3. Tamanho 170 por 320. Define `DISP_DEFAULT`. Exemplo `Arduino1x__S3EMBED__ST7789_8bit`.

```
pinMode(15, OUTPUT); // Alimentação do painel, GPIO 15.
digitalWrite(15, HIGH); // HIGH liga a alimentação.
pinMode(38, OUTPUT); // Luz de fundo, GPIO 38.
digitalWrite(38, HIGH); // HIGH acende a luz.
bus = new Arduino_ESP32PAR8Q(7, 6, 8, 9, 39, 40, 41, 42, 45, 46, 47, 48); // DC, CS, WR, RD e dados D0 a
     D7.
tft = new Arduino_ST7789(bus, 5, rotationScreen, false, 170, 320, 35, 0, 35, 0); // RST 5. Os quatro
    últimos números são deslocamento. Mantenha 35, 0, 35, 0.
tft->begin(); // Esta tela não usa velocidade SPI.
myDisplay.setDrawObject(tft); // Entrega a tela.
```

ST7789 por SPI DMA

Placa ESP32-P4-EYE. Tamanho 240 por 240. Define `DISP_DEFAULT`. A câmera OV2710 vem antes. Veja a seção 6.1.

```
pinMode(20, OUTPUT); // Luz de fundo no GPIO 20.
```

```
digitalWrite(20, LOW); // Nesta placa LOW acende. Se ficar apagada, troque para HIGH.
bus = new Arduino_ESP32SPIDMA(19, 18, 17, 16, -1, SPI2_HOST + 1, false); // DC 19, CS 18, CLK 17, MOSI
    16, MISO -1.
tft = new Arduino_ST7789(bus, 15, rotationScreen, true, 240, 240, 0, 0, 0, 0); // RST 15. IPS true. Os
    quatro zeros são deslocamento.
tft->begin(40000000); // Velocidade desta placa, 40 MHz.
myDisplay.setDrawObject(tft); // Entrega a tela.
```

Telas RGB já soldadas

Define `DISP_DEFAULT`. O `true` do `Arduino_RGB_Display` pede desenho automático. A luz de fundo destes três exemplos é o GPIO 2 em HIGH.

ESP32-8048S043, 800 por 480:

```
bus = new Arduino_ESP32RGBPanel(
  40, 41, 39, 42,                 // DE, VSYNC, HSYNC, PCLK.
  45, 48, 47, 21, 14,             // Vermelho R0 a R4.
  5, 6, 7, 15, 16, 4,             // Verde G0 a G5.
  8, 3, 46, 9, 1,                 // Azul B0 a B4.
  0, 8, 4, 8,                     // Polaridade e tempos horizontais.
  0, 8, 4, 8,                     // Polaridade e tempos verticais.
  1, 16000000, false,             // Clock ativo em nível baixo, 16 MHz, flag de byte.
  0, 0, 0);                       // DE parado, clock parado, buffer de bounce.
tft = new Arduino_RGB_Display(800, 480, bus, rotationScreen, true); // true pede desenho automático.
tft->begin();                     // Liga o painel RGB.
pinMode(2, OUTPUT);               // Luz de fundo no GPIO 2.
digitalWrite(2, HIGH);            // HIGH acende nesta placa.
myDisplay.setDrawObject(tft);     // Entrega a tela.
```

ESP32-8048S070, 800 por 480. Os fios de cor são outros. Não copie o bloco de cima.

```
bus = new Arduino_ESP32RGBPanel(
  41, 40, 39, 42,                 // DE, VSYNC, HSYNC, PCLK.
  14, 21, 47, 48, 45,             // Vermelho R0 a R4.
  9, 46, 3, 8, 16, 1,             // Verde G0 a G5.
  15, 7, 6, 5, 4,                 // Azul B0 a B4.
  0, 180, 30, 16,                 // Tempos horizontais desta placa de 7 pol.
  0, 12, 13, 10,                  // Tempos verticais.
  1, 12000000, false,             // 12 MHz.
  0, 0, 0);                       // DE parado, clock parado, buffer de bounce.
tft = new Arduino_RGB_Display(800, 480, bus, rotationScreen, true); // true pede desenho automático.
tft->begin();                     // Liga o painel RGB.
pinMode(2, OUTPUT);               // Luz de fundo no GPIO 2.
digitalWrite(2, HIGH);            // HIGH acende nesta placa.
myDisplay.setDrawObject(tft);     // Entrega a tela.
```

ESP32-4827S043, 480 por 272. O toque é XPT2046 num SPI separado. Veja a seção 5.1.

```
bus = new Arduino_ESP32RGBPanel(
  40, 41, 39, 42,                 // DE, VSYNC, HSYNC, PCLK.
  45, 48, 47, 21, 14,             // Vermelho R0 a R4.
  5, 6, 7, 15, 16, 4,             // Verde G0 a G5.
  8, 3, 46, 9, 1,                 // Azul B0 a B4.
  0, 1, 1, 43,                    // Tempos horizontais desta placa.
  0, 3, 1, 12,                    // Tempos verticais.
  1, 10000000, false,             // 10 MHz.
  0, 0, 0);                       // DE parado, clock parado, buffer de bounce.
tft = new Arduino_RGB_Display(480, 272, bus, rotationScreen, true); // true pede desenho automático.
tft->begin();                     // Liga o painel RGB.
pinMode(2, OUTPUT);               // Luz de fundo no GPIO 2.
digitalWrite(2, HIGH);            // HIGH acende nesta placa.
myDisplay.setDrawObject(tft);     // Entrega a tela.
```

ST7262 Waveshare, 800 por 480. O exemplo `Arduino1x__S3EMBED__WAVESHARE_ST7262` usa os nomes `DISP_IO_DE` até `DISP_IO_DATA15`. Esses números de GPIO não estão neste repositório. Não troque esses nomes por um pino adivinhado. Os tempos que o exemplo grava são: horizontal 8, 4 e 8, vertical 8, 4 e 8, 16 MHz, buffer

de bounce `800 * 10`. A luz é o GPIO 2 em HIGH. O toque é GT911, SDA 7, SCL 8, INT -1, RST 22.

Telas DSI

Só no ESP32-P4. Define `DISP_DEFAULT`. O `true` do `Arduino_DSI_Display` pede atualização automática. Não é a flag IPS. A tabela `jd9165_init_operations` e as outras vêm da Arduino_GFX ou do cabeçalho da biblioteca. Não escreva a tabela à mão.

JD9165, Guition 7 pol., 1024 por 600. Reset 27. Luz no GPIO 23 em HIGH.

```
bus = new Arduino_ESP32DSIPanel(
  40, 160, 160,                  // Pulso, recuo e avanço horizontal.
  10, 23, 12,                    // Pulso, recuo e avanço vertical.
  48000000);                     // 48 MHz.
tft = new Arduino_DSI_Display(
  1024, 600, bus, rotationScreen, true,
  27, jd9165_init_operations, sizeof(jd9165_init_operations) / sizeof(lcd_init_cmd_t));
tft->begin();                    // Liga o painel DSI.
pinMode(23, OUTPUT);             // Luz de fundo no GPIO 23.
digitalWrite(23, HIGH);          // HIGH acende nesta placa.
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

ST7701, Guition 4 pol., 480 por 800. O exemplo usa rotação 1. Reset 5. Luz no GPIO 23 em HIGH.

```
bus = new Arduino_ESP32DSIPanel(
  12, 42, 42,                    // Tempos horizontais da Guition 4.
  2, 8, 166,                     // Tempos verticais.
  34000000);                     // 34 MHz.
tft = new Arduino_DSI_Display(
  480, 800, bus, rotationScreen, true,
  5, st7701_init_operations, sizeof(st7701_init_operations) / sizeof(lcd_init_cmd_t)); // Reset 5 e
    tabela do ST7701.
tft->begin();                    // Liga o painel DSI.
pinMode(23, OUTPUT);             // Luz de fundo no GPIO 23.
digitalWrite(23, HIGH);          // HIGH acende nesta placa.
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

JD9365, Waveshare 10 pol., 800 por 1280. O exemplo basic_test usa rotação 1. Reset 27. Luz no GPIO 23 em HIGH.

```
bus = new Arduino_ESP32DSIPanel(
  20, 20, 40,                    // Tempos horizontais do JD9365.
  4, 8, 20,                      // Tempos verticais.
  60000000);                     // 60 MHz.
tft = new Arduino_DSI_Display(
  800, 1280, bus, rotationScreen, true,
  27, jd9365_init_operations, sizeof(jd9365_init_operations) / sizeof(lcd_init_cmd_t)); // Reset 27 e
    tabela do JD9365.
tft->begin();                    // Liga o painel DSI.
pinMode(23, OUTPUT);             // Luz de fundo no GPIO 23.
digitalWrite(23, HIGH);          // HIGH acende nesta placa.
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

EK79007, Functional EV Board, 1024 por 600. Reset no pino `DISP_RST` do exemplo da câmera.

```
bus = new Arduino_ESP32DSIPanel(
  10, 160, 160,                  // Tempos horizontais do EK79007.
  1, 23, 12,                     // Tempos verticais.
  52000000);                     // 52 MHz.
tft = new Arduino_DSI_Display(
  1024, 600, bus, rotationScreen, true,
  DISP_RST, ek79007_init_operations, sizeof(ek79007_init_operations) / sizeof(lcd_init_cmd_t));
tft->begin();                    // Liga o painel DSI.
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

ILI9881C, Waveshare 7 pol. com câmera, 720 por 1280. Antes do painel, ligue o chip auxiliar. Os tempos e os pinos I2C estão em `src/displays/ili9881c_waveshare_7in.h`. Reset do exemplo: -1. SDA 7. SCL 8.

```
if (!myDisplay.waveshareDsiAuxBegin(7, 8)) { // I2C do chip auxiliar. SDA 7, SCL 8.
  Serial.println("Controlador auxiliar do painel falhou");
}
bus = new Arduino_ESP32DSIPanel(
  ILI9881C_WAVESHARE_7_HSYNC_PULSE, ILI9881C_WAVESHARE_7_HSYNC_BACK, ILI9881C_WAVESHARE_7_HSYNC_FRONT,
  ILI9881C_WAVESHARE_7_VSYNC_PULSE, ILI9881C_WAVESHARE_7_VSYNC_BACK, ILI9881C_WAVESHARE_7_VSYNC_FRONT,
  ILI9881C_WAVESHARE_7_PIXEL_CLOCK_HZ, ILI9881C_WAVESHARE_7_LANE_MBPS); // 80 MHz e 1000 Mbps. Não
    troque estes nomes.
tft = new Arduino_DSI_Display(
  720, 1280, bus, rotationScreen, true,
  -1, ili9881c_waveshare_7_init_operations, sizeof(ili9881c_waveshare_7_init_operations) /
    sizeof(lcd_init_cmd_t));
tft->begin();                    // Liga o painel DSI.
myDisplay.waveshareDsiAuxBacklight(255); // Brilho de 0 a 255 no chip auxiliar.
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

PCD8544, SSD1306 e ST7565

O catálogo escreve PDC8544 e SD1306. Na biblioteca os defines são `DISP_PCD8544` e `DISP_SSD1306`. A ST7565 usa `DISP_U8G2`.

PCD8544, 84 por 48. Pinos do exemplo: CLK 18, DIN 23, DC 2, CS 5, RST 4. Depois de desenhar, chame `tft->display()`.

```
tft = new Adafruit_PCD8544(18, 23, 2, 5, 4); // CLK, DIN, DC, CS, RST.
tft->begin();                    // Liga o Nokia 5110.
tft->setContrast(75);            // Contraste de 0 a 127. 75 é o do exemplo.
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

SSD1306, 128 por 32. Endereço 0x3C. SDA 21. SCL 22. RST -1. Depois de desenhar, chame `tft->display()`.

```
Wire.begin(21, 22);              // SDA 21, SCL 22.
tft = new Adafruit_SSD1306(128, 32, &Wire, -1); // Largura, altura, I2C, reset -1.
if (!tft->begin(SSD1306_SWITCHCAPVCC, 0x3C, false, false)) { // Endereço 0x3C. Algumas placas usam 0x3D.
  Serial.println("Falha ao iniciar o SSD1306"); // Fio ou endereço errado.
  for (;;) { }                   // Para aqui. Corrija o fio e grave de novo.
}
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

ST7565, 128 por 64. CLK 18, DT 23, CS 5, DC 2, RST 4. Depois de desenhar, chame `tft->sendBuffer()`.

```
tft = new U8G2_ST7565_NHD_C12864_F_4W_SW_SPI(U8G2_R0, 18, 23, 5, 2, 4); // Rotação U8g2, CLK, DT, CS,
    DC, RST.
tft->begin();                    // Liga a ST7565.
tft->enableUTF8Print();          // Permite texto UTF-8.
tft->clearBuffer();              // Limpa o buffer antes do primeiro desenho.
myDisplay.setDrawObject(tft);    // Entrega a tela.
```

5. Toques suportados

Chip	Define em `user_setup.h`	Fios	Calibrar	Função no sketch
GT911	`TOUCH_GT911`	I2C: SDA, SCL, INT, RST	não	`startTouchGT911`
GT911 Waveshare	`TOUCH_GT911_WAVESHARE`	I2C: SDA, SCL	não	`startTouchGT911Waveshare`
XPT2046	`TOUCH_XPT2046`	SPI: CS, CLK, MOSI, MISO	sim	`startTouchXPT2046`
FT6236	`TOUCH_FT6236U`	I2C: SDA, SCL	não	`startTouchFT6236U`
FT6336	`TOUCH_FT6336`	I2C: SDA, SCL, INT, RST	não	`startTouchFT6336`
CST816	`TOUCH_CST816`	I2C	não	`startTouchCST816`
CST820	`TOUCH_CST820`	I2C	não	`startTouchCST820`
GSL3680	`TOUCH_GSL3680`	I2C: SDA, SCL, INT, RST	não	`startTouchGSL3680`

A ordem da chamada é esta:

• `setTouchCorners`

• `setInvertAxis`

• `setSwapAxis`

• `startTouch...`

Os três primeiros guardam o ajuste. O `startTouch` copia esse ajuste. Mude os três antes do `startTouch`.

No toque I2C, a ordem dos pinos na função é: largura, altura, rotação, **SDA, SCL**, INT, RST. Onde o chip não tem INT ou RST, o exemplo usa `-1`.

5.1 Como configurar cada toque

Não chame `startTouch(largura, altura, rotação, spi)`. Essa função genérica não está implementada. Ela só escreve um erro no log. Use a função do chip.

O catálogo chama o FT6336 de `TOUCH_FT6336U`. Na biblioteca o define que compila é `TOUCH_FT6336`. A função é `startTouchFT6336`.

Antes de todo `startTouch...`, chame estas três linhas. Elas guardam o mapa. O `startTouch` copia esse mapa. Se você chamar as três depois, o toque ignora o ajuste.

```
myDisplay.setTouchCorners(0, DISPLAY_W, 0, DISPLAY_H); // x0, x1, y0, y1. x1 é a largura. y1 é a altura.
myDisplay.setInvertAxis(false, false); // O primeiro valor espelha X. O segundo espelha Y.
myDisplay.setSwapAxis(false); // true troca o eixo X com o eixo Y.
```

Só um define de toque entra em `user_setup.h`. A função do outro chip nem é compilada.

GT911

Define `TOUCH_GT911`. Função `startTouchGT911(largura, altura, rotação, SDA, SCL, INT, RST)`.

Na Guition 7 e na Guition 4 o exemplo usa SDA 7, SCL 8. Na Guition 7, INT é -1 e RST é 22. Na Guition 4, INT e RST são -1.

Nas placas ESP32-S3 8048S043 e 8048S070 o exemplo usa SDA 19, SCL 20, INT -1 e RST 38.

```
myDisplay.startTouchGT911(DISPLAY_W, DISPLAY_H, rotationScreen, 7, 8, -1, 22); // SDA, SCL, INT, RST.
    Troque os quatro últimos números.
```

GT911 Waveshare

Define `TOUCH_GT911_WAVESHARE`. Esta variação não recebe INT nem RST. O exemplo da Waveshare 7 pol. usa SDA 7 e SCL 8.

```
myDisplay.startTouchGT911Waveshare(DISPLAY_W, DISPLAY_H, rotationScreen, 7, 8); // SDA 7, depois SCL 8.
```

XPT2046

Define `TOUCH_XPT2046`. Este toque pede calibração. Depois do `startTouchXPT2046`, chame `myDisplay.checkCalibration()`. Na primeira vez a pessoa toca os cantos. O valor fica gravado. Para calibrar de novo, chame `myDisplay.recalibrate()` uma vez, grave, teste e apague essa linha.

Há duas funções com o mesmo nome. Prefira a que recebe o objeto SPI já aberto. A outra cria um segundo SPI nos mesmos fios.

Forma com SPI compartilhado. O último número é o CS da tela, para a biblioteca soltar a tela antes de ler o dedo. Exemplo ILI9488 no ESP32-S3: CLK 12, MISO 18, MOSI 11, CS da tela 10, CS do toque 14, velocidade do toque 2500000, velocidade da tela 27000000.

```
spi_shared.begin(12, 18, 11); // CLK, MISO, MOSI. O mesmo objeto vai para a tela e para o toque.
myDisplay.startTouchXPT2046(DISPLAY_W, DISPLAY_H, rotationScreen, 14, &spi_shared, tft, 2500000,
    27000000, 10); // CS do toque, SPI, tela, Hz do toque, Hz da tela, CS da tela.
myDisplay.checkCalibration(); // Na primeira vez, toque nos cantos.
```

Forma com os fios soltos. O exemplo ILI9488 de 240 por 320 usa CLK 18, MOSI 23, MISO 19 e CS do toque 21. Os dois últimos argumentos podem ficar de fora. O padrão é velocidade 0 e CS da tela -1.

```
myDisplay.startTouchXPT2046(DISPLAY_W, DISPLAY_H, rotationScreen, 18, 23, 19, 21, tft, 2500000); // CLK,
     MOSI, MISO, CS do toque, tela, Hz do toque.
myDisplay.checkCalibration();
```

Na placa 4827S043 o toque usa CS 38, CLK 12, MOSI 11 e MISO 13, no objeto `spi_shared`, com 2500000 Hz. A tela dessa placa é RGB e não divide o CS.

FT6236

Define `TOUCH_FT6236U`. Função `startTouchFT6236U(largura, altura, rotação, SDA, SCL, INT, RST)`. O exemplo de 3,5 pol. usa SDA 21, SCL 22 e -1 em INT e RST.

```
myDisplay.startTouchFT6236U(DISPLAY_W, DISPLAY_H, rotationScreen, 21, 22, -1, -1); // SDA, SCL, INT,
    RST.
```

FT6336

Define `TOUCH_FT6336`. Função `startTouchFT6336(largura, altura, rotação, SDA, SCL, INT, RST)`. O exemplo de 4 pol. usa SDA 21, SCL 22, INT 19 e RST 15.

```
myDisplay.startTouchFT6336(DISPLAY_W, DISPLAY_H, rotationScreen, 21, 22, 19, 15); // SDA, SCL, INT, RST.
```

CST820

Define `TOUCH_CST820`. Função `startTouchCST820(largura, altura, rotação, SDA, SCL, INT, RST)`. A Guition 2.8 usa SDA 33, SCL 32, INT 21 e RST 25.

```
myDisplay.startTouchCST820(DISPLAY_W, DISPLAY_H, rotationScreen, 33, 32, 21, 25); // SDA, SCL, INT, RST.
```

CST816

Define `TOUCH_CST816`. Função `startTouchCST816(largura, altura, rotação, SDA, SCL, INT, RST)`. O catálogo não traz pinos. Esta biblioteca não tem exemplo com GPIO deste chip. Não invente o número. Use -1 em INT ou RST quando o fio não existir.

```
myDisplay.startTouchCST816(DISPLAY_W, DISPLAY_H, rotationScreen, -1, -1, -1, -1); // Ordem: SDA, SCL,
    INT, RST. Troque cada -1 pelo GPIO da placa. Não deixe -1 no SDA nem no SCL.
```

GSL3680

Define `TOUCH_GSL3680`. Função `startTouchGSL3680(largura, altura, rotação, SDA, SCL, INT, RST)`. A Waveshare de 10 pol. usa SDA 7, SCL 8, INT 21 e RST 22.

```
myDisplay.startTouchGSL3680(DISPLAY_W, DISPLAY_H, rotationScreen, 7, 8, 21, 22); // SDA, SCL, INT, RST.
```

6. Câmeras suportadas

A câmera entra só no ESP32-P4. O define fica em `user_setup.h`.

Sensor	Define	Tamanho que você pode pedir
OV5647	`CAM_OV5647`	800x1280, 800x640, 800x800, 1920x1080, 1280x960. Use 0 e 0 para o padrão do driver.
OV02C10	`CAM_OV02C10`	O driver usa a tabela interna. Largura e altura diferentes de zero são ignoradas.
SC2336	`CAM_SC2336`	O driver usa a tabela interna. Largura e altura diferentes de zero são ignoradas.
OV2710	`CAM_OV2710`	O driver usa a tabela interna. A placa EYE ainda manda 1920x1080.

Ordem no `setup`:

- `setRotationCamera(rotação, espelhoX, espelhoY)`
- `startCamera(...)`

• Se `startCamera` devolver verdadeiro, chame `getCamera().startPpa()`

Na ESP32-P4-EYE a câmera também precisa de energia e de clock antes do `startCamera`. O exemplo 8.11 mostra isso.

Na maior parte das placas P4, a câmera e o toque usam os mesmos fios **SDA = 7** e **SCL = 8**. A biblioteca divide esse I2C. Deixe esses dois pinos com a biblioteca.

6.1 Como iniciar cada câmera

A câmera só entra no ESP32-P4. O sketch inclui `<ESP_Video.h>`. Só um define de câmera entra em `user_setup.h`.

A ordem, nas placas que não são a EYE, é esta. Chame antes de criar o painel.

```
myDisplay.setRotationCamera(rotationScreen, false, false); // Rotação de 0 a 3, espelho horizontal,
    espelho vertical.
if (myDisplay.startCamera(0, 8, 7, 400000, -1, -1, 1920, 1080)) { // Porta I2C, SCL, SDA, Hz do I2C,
    reset, pwdn, largura, altura.
  myDisplay.getCamera().startPpa(); // Liga o acelerador que reduz a imagem para caber na tela.
} else {
  Serial.println("Camera nao iniciou"); // A tela pode continuar. Confira o define e os fios 7 e 8.
}
```

O sexto número é o pino pwdn. Os exemplos entregam `CAM_INT` nesse lugar. Quando a interrupção não existe, o valor é -1. O quinto número é o reset. Na maior parte das placas também é -1.

`setCameraPreviewFps(numero)` aceita de 5 a 24. Esse número é a taxa da miniatura. É opcional.

OV5647

Define `CAM_OV5647`. Você pode pedir 800x1280, 800x640, 800x800, 1920x1080 ou 1280x960. O par do exemplo é 1920 por 1080. Use 0 e 0 para o padrão do driver. SDA 7. SCL 8. I2C 400000. Porta 0.

O bloco comum acima já é o início desta câmera. Não acrescente energia nem clock.

OV02C10

Define `CAM_OV02C10`. O driver usa a tabela interna. Largura e altura diferentes de zero são ignoradas. O exemplo ainda chama `startCamera` com 1920 e 1080. SDA 7. SCL 8. I2C 400000. Porta 0. Reset -1. Pwdn -1.

Use o mesmo bloco da OV5647. Mude só o define em `user_setup.h`.

SC2336

Define `CAM_SC2336`. O driver também ignora largura e altura diferentes de zero. O exemplo da Functional EV Board usa os mesmos fios: SDA 7, SCL 8, I2C 400000, porta 0, reset -1, pwdn -1.

Use o mesmo bloco da OV5647. Mude só o define em `user_setup.h`.

OV2710

Define `CAM_OV2710`. Esta câmera é a da ESP32-P4-EYE. Ela precisa de energia e de clock antes do `startCamera`. O nome `CAM_FREQUENCY_SCCB` no exemplo vale 24000000, mas esse número é o clock do sensor, não o I2C. O I2C entra como 400000.

Pinos do exemplo: alimentação no GPIO 12, clock no GPIO 11, SDA 14, SCL 13, reset 26, pwdn -1, imagem 1920 por 1080, porta I2C 0.

```
if (!myDisplay.getCamera().enableBoardPower(12)) { // GPIO 12 liga a alimentação da câmera e da tela.
  Serial.println("Falha ao ligar a energia da camera");
  return; // Sem energia o restante do setup não continua.
}
if (!myDisplay.getCamera().startXclk(11, 24000000)) { // Clock de 24 MHz no GPIO 11.
  return;
}
myDisplay.setRotationCamera(rotationScreen, false, false); // Rotação e espelho, antes de startCamera.
if (myDisplay.startCamera(0, 13, 14, 400000, 26, -1, 1920, 1080)) { // Porta 0, SCL 13, SDA 14, I2C
    400000, reset 26, pwdn -1.
  myDisplay.getCamera().startPpa();
} else {
  Serial.println("Camera nao iniciou");
}
```

7. Placas prontas

`cam = 0` quer dizer sem câmera no cadastro da placa. `touch = 0` quer dizer sem toque. Você ainda pode acrescentar uma câmera no `user_setup.h` quando o hardware tiver o sensor. O exemplo da Guition 7 deste guia faz isso.

Placa	Tamanho	Tela	Toque	Câmera no cadastro	Chip
ESP32-S3 4.3 pol. 8048	800x480	ESP32-8048S043	GT911	nenhuma	ESP32-S3
ESP32-S3 4.3 pol.	480x272	ESP32-4827S043	XPT2046	nenhuma	ESP32-S3
ESP32-S3 7 pol.	800x480	ESP32-8048S070	GT911	nenhuma	ESP32-S3
TFT redondo GC9A01	240x240	GC9A01	nenhum	nenhuma	ESP32
TFT 1.44 pol.	128x128	ST7735	nenhum	nenhuma	ESP32
ST7789 quadrado	240x240	ST7789 SPI	nenhum	nenhuma	ESP32
ST7735 1.8 pol.	128x160	ST7735	nenhum	nenhuma	ESP32
ILI9225 2.2 pol.	176x220	ILI9225	nenhum	nenhuma	ESP32
ILI9488 2.8 pol.	240x320	ILI9488	XPT2046	nenhuma	ESP32
ST7789 2 pol. azul	240x320	ST7789 SPI	nenhum	nenhuma	ESP32
ILI9488 3.5 pol.	480x320	ILI9488	FT6236	nenhuma	ESP32
ILI9488 3.5 pol.	320x480	ILI9488	XPT2046	nenhuma	ESP32
ILI9488 4 pol.	320x480	ILI9488	FT6336	nenhuma	ESP32
ILI9488 4 pol.	320x480	ILI9488	XPT2046	nenhuma	ESP32
ST7789 8 bits	170x320	ST7789 8 bits	nenhum	nenhuma	ESP32-S3
Nokia 5110	84x48	PCD8544	nenhum	nenhuma	ESP32

Placa	Tamanho	Tela	Toque	Câmera no cadastro	Chip
OLED	128x32	SSD1306	nenhum	nenhuma	ESP32
ST7565	128x64	ST7565	nenhum	nenhuma	ESP32
Waveshare 4.3 pol.	800x480	ST7262	GT911	nenhuma	ESP32-P4
Waveshare 10.1 pol.	800x1280	JD9365	GSL3680	OV02C10	ESP32-P4
Lousa 3.5 pol.	320x480	ST7796	FT6336	nenhuma	ESP32
Guition 7 pol.	1024x600	JD9165	GT911	nenhuma no cadastro	ESP32-P4
Guition 2.8 pol.	240x320	ST7789 da placa	CST820	nenhuma	ESP32
Guition 4 pol.	480x800	ST7701	GT911	OV02C10	ESP32-P4
Waveshare DSI com câmera	720x1280	ILI9881C	GT911 Waveshare	OV5647	ESP32-P4
Functional EV Board	1024x600	EK79007	GT911	SC2336	ESP32-P4
ESP32-P4-EYE	240x240	ST7789 SPI DMA	nenhum	OV2710	ESP32-P4

8. Exemplos de combinações

Cada exemplo é um sketch curto. Antes de compilar, grave o `user_setup.h` indicado. Os pinos vieram dos exemplos da biblioteca. Se a sua placa usa outros GPIO, troque só o número do pino.

8.1 Tela SPI sem toque e sem câmera

Placa ESP32. Tela ST7789, 240 por 240.

`user_setup.h`:

```
#define DISP_DEFAULT // Tela colorida. Não há linha de toque nem de câmera.
```

Sketch:

```
#include <SPI.h>                  // Biblioteca do barramento SPI.
#include <Arduino_GFX_Library.h>  // Desenho da tela.
#include <displaywx.h>            // Esta biblioteca.

const int DISPLAY_W = 240;        // Largura da tela em pixels. Mude se a sua tela for outra.
const int DISPLAY_H = 240;        // Altura da tela em pixels.
const int DISP_FREQUENCY = 27000000; // Velocidade do SPI. Se a imagem falhar, baixe para 20000000.
const int DISP_MOSI = 23;         // Fio de dados da tela.
const int DISP_MISO = -1;         // Esta tela não devolve dados. -1 desliga o fio.
const int DISP_SCLK = 18;         // Relógio do SPI.
const int DISP_CS = 5;            // Seleção da tela.
const int DISP_DC = 2;            // Escolhe comando ou dado.
const int DISP_RST = 4;           // Reset da tela. Use -1 se o pino não existir.
const uint8_t rotationScreen = 0; // 0, 1, 2 ou 3. Veja a seção 9.
```

```
const bool isIPS = false;         // true em alguns painéis IPS. Veja a seção 9.

#if defined(CONFIG_IDF_TARGET_ESP32S3)
SPIClass spi_shared(FSPI);        // No ESP32-S3 o SPI da tela usa FSPI.
#else
SPIClass spi_shared(HSPI);        // No ESP32 clássico o SPI da tela usa HSPI.
#endif
Arduino_DataBus *bus = nullptr;   // O objeto do barramento nasce no setup.
Arduino_GFX *tft = nullptr;       // O objeto da tela nasce no setup.
DisplayWX myDisplay;              // O objeto principal da biblioteca.

void screen0();                   // A função que pinta a tela.
void loadWidgets();               // A função que configura os widgets. Aqui fica vazia.

void setup() {
  Serial.begin(115200);           // Abre a serial para mensagens.
  spi_shared.begin(DISP_SCLK, DISP_MISO, DISP_MOSI); // Liga o SPI nos três fios.
  bus = new Arduino_HWSPI(DISP_DC, DISP_CS, DISP_SCLK, DISP_MOSI, DISP_MISO, &spi_shared); // Liga o
    barramento da tela a esse SPI.
  tft = new Arduino_ST7789(bus, DISP_RST, rotationScreen, isIPS); // Cria a tela ST7789.
  tft->begin(DISP_FREQUENCY);     // Inicia a tela na velocidade escolhida.
  myDisplay.setDrawObject(tft);   // Entrega a tela para a DisplayWX.
  loadWidgets();                  // Prepara os widgets. Neste exemplo não há nenhum.
  myDisplay.loadScreen(screen0);  // Mostra a primeira tela.
  myDisplay.createTask(false, 3); // Liga a tarefa que desenha. false desliga o watchdog. 3 é o tempo em
     segundos se você ligar o watchdog.
}

void loop() {
}                                 // O desenho acontece na tarefa. O loop pode ficar vazio.

void screen0() {
  myDisplay.setBackgroundColor(CFK_WHITE); // Pinta o fundo e guarda a cor para apagar texto. Troque
    CFK_WHITE por outra cor CFK_.
  myDisplay.drawWidgetsOnScreen(0); // Desenha os widgets da tela 0.
}

void loadWidgets() {
}                                 // Sem widgets neste exemplo.
```

8.2 Tela SPI e toque XPT2046 no mesmo SPI

Placa ESP32-S3. Tela ILI9488, 240 por 320. O toque usa os mesmos fios CLK, MOSI e MISO. O CS da tela e o CS do toque são pinos diferentes.

`user_setup.h`:

```
#define DISP_DEFAULT   // Tela colorida.
#define TOUCH_XPT2046  // Toque resistivo no SPI.

#include <SPI.h>
#include <Arduino_GFX_Library.h>
#include <displaywx.h>

const int DISPLAY_W = 240;          // Largura usada também pelo mapa do toque.
const int DISPLAY_H = 320;          // Altura usada também pelo mapa do toque.
const int DISP_FREQUENCY = 27000000; // Velocidade do SPI da tela.
const int TOUCH_FREQUENCY = 2500000; // Velocidade do SPI do toque. Mais baixa que a da tela.
const int TOUCH_MAP_X0 = 0;          // Canto esquerdo do toque.
const int TOUCH_MAP_X1 = 240;        // Canto direito do toque. Use a largura da tela.
const int TOUCH_MAP_Y0 = 0;          // Canto de cima do toque.
const int TOUCH_MAP_Y1 = 320;        // Canto de baixo do toque. Use a altura da tela.
const bool TOUCH_SWAP_XY = false;    // true troca X com Y. Veja a seção 9.
const bool TOUCH_INVERT_X = false;   // true inverte a esquerda com a direita.
const bool TOUCH_INVERT_Y = false;   // true inverte a parte de cima com a de baixo.
const int DISP_MOSI = 11;            // Dado, compartilhado com o toque.
const int DISP_MISO = 18;            // Dado de volta, compartilhado com o toque.
const int DISP_SCLK = 12;            // Relógio, compartilhado com o toque.
const int DISP_CS = 10;              // CS só da tela.
const int DISP_DC = 8;               // Dado ou comando, só da tela.
```

```
const int DISP_RST = 9;              // Reset só da tela.
const int TOUCH_CS = 14;             // CS só do toque.
const uint8_t rotationScreen = 0;    // Rotação da tela e do toque.
const bool isIPS = false;            // Flag IPS deste painel.

#if defined(CONFIG_IDF_TARGET_ESP32S3)
SPIClass spi_shared(FSPI);           // Um único objeto SPI para a tela e para o toque.
#else
SPIClass spi_shared(HSPI);
#endif
Arduino_DataBus *bus = nullptr;
Arduino_GFX *tft = nullptr;
DisplayWX myDisplay;

void screen0();
void loadWidgets();

void setup() {
  Serial.begin(115200);
  spi_shared.begin(DISP_SCLK, DISP_MISO, DISP_MOSI); // Abre o SPI uma vez.
  bus = new Arduino_HWSPI(DISP_DC, DISP_CS, DISP_SCLK, DISP_MOSI, DISP_MISO, &spi_shared); // A tela usa
     esse objeto.
  tft = new Arduino_ILI9488_18bit(bus, DISP_RST, rotationScreen, isIPS); // Modelo ILI9488 em 18 bits.
  tft->begin(DISP_FREQUENCY);
  myDisplay.setDrawObject(tft);
  myDisplay.setTouchCorners(TOUCH_MAP_X0, TOUCH_MAP_X1, TOUCH_MAP_Y0, TOUCH_MAP_Y1); // Área útil do
    toque.
  myDisplay.setInvertAxis(TOUCH_INVERT_X, TOUCH_INVERT_Y); // Espelho X e Y.
  myDisplay.setSwapAxis(TOUCH_SWAP_XY); // Troca dos eixos.
  myDisplay.startTouchXPT2046(DISPLAY_W, DISPLAY_H, rotationScreen, TOUCH_CS, &spi_shared, tft,
    TOUCH_FREQUENCY, DISP_FREQUENCY, DISP_CS); // O último argumento é o CS da tela, para a biblioteca
    soltar a tela antes de ler o toque.
  myDisplay.checkCalibration(); // Na primeira vez, toque nos cantos que a tela pedir. O valor fica
    gravado.
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}

void loop() {
}

void screen0() {
  myDisplay.setBackgroundColor(CFK_WHITE); // Pinta o fundo e guarda a cor. Troque CFK_WHITE por outra
    cor CFK_.
  myDisplay.drawWidgetsOnScreen(0);
}

void loadWidgets() {
}
```

Existe outra forma de `startTouchXPT2046` que recebe CLK, MOSI e MISO em vez do objeto SPI. Essa forma cria um segundo SPI nos mesmos fios. Prefira a forma com `&spi_shared`, como no exemplo acima.

Para calibrar de novo, chame `myDisplay.recalibrate()` uma vez no `setup`, grave, teste, e depois apague essa linha.

8.3 Tela SPI e toque I2C

Placa ESP32. Tela ILI9488, 480 por 320. Toque FT6236 nos fios I2C. A tela e o toque não dividem fios.

`user_setup.h`:

```
#define DISP_DEFAULT
#define TOUCH_FT6236U

#include <SPI.h>
#include <Arduino_GFX_Library.h>
```

```
#include <displaywx.h>

const int DISPLAY_W = 480;
const int DISPLAY_H = 320;
const int DISP_FREQUENCY = 27000000;
const int TOUCH_MAP_X0 = 0;
const int TOUCH_MAP_X1 = 480;       // Igual à largura.
const int TOUCH_MAP_Y0 = 0;
const int TOUCH_MAP_Y1 = 320;       // Igual à altura.
const bool TOUCH_SWAP_XY = false;
const bool TOUCH_INVERT_X = false;
const bool TOUCH_INVERT_Y = false;
const int DISP_MOSI = 23;
const int DISP_MISO = -1;
const int DISP_SCLK = 18;
const int DISP_CS = 5;
const int DISP_DC = 2;
const int DISP_RST = 4;
const int TOUCH_SCL = 22;           // Relógio I2C do toque.
const int TOUCH_SDA = 21;           // Dado I2C do toque.
const uint8_t rotationScreen = 0;
const bool isIPS = false;

#if defined(CONFIG_IDF_TARGET_ESP32S3)
SPIClass spi_shared(FSPI);
#else
SPIClass spi_shared(HSPI);
#endif
Arduino_DataBus *bus = nullptr;
Arduino_GFX *tft = nullptr;
DisplayWX myDisplay;
void screen0();
void loadWidgets();

void setup() {
  Serial.begin(115200);
  spi_shared.begin(DISP_SCLK, DISP_MISO, DISP_MOSI);
  bus = new Arduino_HWSPI(DISP_DC, DISP_CS, DISP_SCLK, DISP_MOSI, DISP_MISO, &spi_shared);
  tft = new Arduino_ILI9488_18bit(bus, DISP_RST, rotationScreen, isIPS);
  tft->begin(DISP_FREQUENCY);
  myDisplay.setDrawObject(tft);
  myDisplay.setTouchCorners(TOUCH_MAP_X0, TOUCH_MAP_X1, TOUCH_MAP_Y0, TOUCH_MAP_Y1);
  myDisplay.setInvertAxis(TOUCH_INVERT_X, TOUCH_INVERT_Y);
  myDisplay.setSwapAxis(TOUCH_SWAP_XY);
  myDisplay.startTouchFT6236U(DISPLAY_W, DISPLAY_H, rotationScreen, TOUCH_SDA, TOUCH_SCL, -1, -1); //
    SDA, depois SCL. -1 porque este exemplo não usa INT nem RST.
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}

void loop() {
}

void screen0() {
  myDisplay.setBackgroundColor(CFK_WHITE); // Pinta o fundo e guarda a cor. Troque CFK_WHITE por outra
    cor CFK_.
  myDisplay.drawWidgetsOnScreen(0);
}

void loadWidgets() {
}
```

Para o FT6336, troque o define para `TOUCH_FT6336` e a chamada para `startTouchFT6336(largura, altura, rotação, SDA, SCL, INT, RST)`.

8.4 Tela I2C sem toque

Placa ESP32. OLED SSD1306, 128 por 32.

`user_setup.h`:

```
#define DISP_SSD1306

#include <SPI.h>
#include <Adafruit_GFX.h>       // Base de desenho da Adafruit.
#include <Adafruit_SSD1306.h>   // Driver do OLED.
#include <displaywx.h>

const int DISPLAY_W = 128;
const int DISPLAY_H = 32;
const uint8_t DISP_ADDRESS = 0x3C; // Endereço I2C. Algumas placas usam 0x3D.
const int DISP_SDA = 21;
const int DISP_SCL = 22;
const int DISP_RST = -1;           // -1 quando o reset está ligado no circuito.
const uint8_t rotationScreen = 0;
Adafruit_SSD1306 *tft = nullptr;
DisplayWX myDisplay;
void screen0();
void loadWidgets();

void setup() {
  Serial.begin(115200);
  Wire.begin(DISP_SDA, DISP_SCL);  // Abre o I2C da tela.
  tft = new Adafruit_SSD1306(128, 32, &Wire, DISP_RST); // Largura, altura, I2C e reset.
  if (!tft->begin(SSD1306_SWITCHCAPVCC, DISP_ADDRESS, false, false)) { // Liga o OLED.
    Serial.println("Falha ao iniciar o SSD1306"); // Mensagem se o fio ou o endereço estiver errado.
    for (;;) { }                          // Para aqui. Corrija o fio e grave de novo.
  }
  myDisplay.setDrawObject(tft);
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}

void loop() {
}

void screen0() {
  myDisplay.setBackgroundColor(CFK_WHITE); // Pinta o buffer e guarda a cor. Troque CFK_WHITE por outra
    cor CFK_.
  myDisplay.drawWidgetsOnScreen(0);
  tft->display();                      // Envia o buffer para o vidro. O OLED só muda depois desta
    linha.
}

void loadWidgets() {
}
```

Nesta tela, botões, gráficos e caixa de texto ficam desligados. Use Label, Led e VBar.

8.5 ESP32-S3 com tela RGB e toque GT911

Placa com a tela 800 por 480 já soldada. Os pinos abaixo são os da placa 8048S043.

`user_setup.h`:

```
#define DISP_DEFAULT
#define TOUCH_GT911

#include <SPI.h>
#include <Arduino_GFX_Library.h>
#include <displaywx.h>

const int DISPLAY_W = 800;
const int DISPLAY_H = 480;
const int TOUCH_MAP_X0 = 0;
const int TOUCH_MAP_X1 = 800;
const int TOUCH_MAP_Y0 = 0;
```

```
const int TOUCH_MAP_Y1 = 480;
const bool TOUCH_SWAP_XY = false;
const bool TOUCH_INVERT_X = false;
const bool TOUCH_INVERT_Y = false;
const int TOUCH_SCL = 20;
const int TOUCH_SDA = 19;
const int TOUCH_INT = -1;
const int TOUCH_RST = 38;
const uint8_t rotationScreen = 0;
const bool isIPS = false;
#define DISP_LED 2                    // Luz de fundo desta placa.
Arduino_ESP32RGBPanel *bus = nullptr;
Arduino_RGB_Display *tft = nullptr;
DisplayWX myDisplay;
void screen0();
void loadWidgets();

void setup() {
  Serial.begin(115200);
  bus = new Arduino_ESP32RGBPanel(
    40, 41, 39, 42,                 // DE, VSYNC, HSYNC, PCLK.
    45, 48, 47, 21, 14,             // Bits vermelhos R0 a R4.
    5, 6, 7, 15, 16, 4,             // Bits verdes G0 a G5.
    8, 3, 46, 9, 1,                 // Bits azuis B0 a B4.
    0, 8, 4, 8,                     // Polaridade e tempos horizontais.
    0, 8, 4, 8,                     // Polaridade e tempos verticais.
    1, 16000000, isIPS,             // Clock ativo em nível baixo, velocidade, flag de byte.
    0, 0, 0);                       // DE parado, clock parado, buffer de bounce.
  tft = new Arduino_RGB_Display(800, 480, bus, rotationScreen, true); // true pede o desenho automático.
  tft->begin();
  pinMode(DISP_LED, OUTPUT);        // A luz de fundo é uma saída.
  digitalWrite(DISP_LED, HIGH);     // HIGH acende a luz desta placa.
  myDisplay.setDrawObject(tft);
  myDisplay.setTouchCorners(TOUCH_MAP_X0, TOUCH_MAP_X1, TOUCH_MAP_Y0, TOUCH_MAP_Y1);
  myDisplay.setInvertAxis(TOUCH_INVERT_X, TOUCH_INVERT_Y);
  myDisplay.setSwapAxis(TOUCH_SWAP_XY);
  myDisplay.startTouchGT911(DISPLAY_W, DISPLAY_H, rotationScreen, TOUCH_SDA, TOUCH_SCL, TOUCH_INT,
    TOUCH_RST);
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}

void loop() {
}

void screen0() {
  myDisplay.setBackgroundColor(CFK_WHITE); // Pinta o fundo e guarda a cor. Troque CFK_WHITE por outra
    cor CFK_.
  myDisplay.drawWidgetsOnScreen(0);
}

void loadWidgets() {
}
```

8.6 ESP32-P4, Guition 7 pol., GT911 e câmera OV5647

Esta é a combinação base dos widgets. O código completo, comentado linha a linha, está na seção 10.

Resumo:

• `user_setup.h`: `DISP_DEFAULT`, `TOUCH_GT911`, `CAM_OV5647`

• Tela JD9165, 1024 por 600, DSI

• Toque: SDA 7, SCL 8, RST 22

• Câmera no mesmo I2C: SDA 7, SCL 8, imagem 1920 por 1080

• Luz de fundo no GPIO 23

• Reset da tela no GPIO 27

8.7 ESP32-P4, Waveshare 10 pol., GSL3680 e câmera OV02C10

`user_setup.h`:

```
#define DISP_DEFAULT
#define TOUCH_GSL3680
#define CAM_OV02C10
```

Copie o sketch da seção 10 e troque os números que já existem. Mantenha um único `const` de cada nome. O `setup` abaixo entra no lugar do `setup` da seção 10. Os includes, `screen0` e `loop` continuam os mesmos.

```
const int DISPLAY_W = 800;          // Largura desta tela.
const int DISPLAY_H = 1280;         // Altura desta tela.
const int TOUCH_MAP_X1 = 800;       // Mapa do toque igual à tela.
const int TOUCH_MAP_Y1 = 1280;
const int TOUCH_INT = 21;           // Interrupção do GSL3680.
const int TOUCH_RST = 22;           // Reset do toque.
const bool isIPS = true;            // Este painel usa a flag IPS.
const int CAM_SDA = 7;              // Mesmo SDA do toque.
const int CAM_SCL = 8;              // Mesmo SCL do toque.

void setup() {
  Serial.begin(115200);
  myDisplay.setRotationCamera(rotationScreen, false, false); // Rotação da câmera, sem espelho.
  if (myDisplay.startCamera(0, CAM_SCL, CAM_SDA, 400000, -1, -1, 1920, 1080)) { // Porta I2C 0. A
    OV02C10 ignora 1920x1080 e usa a tabela interna.
    myDisplay.getCamera().startPpa(); // Liga o acelerador que reduz a imagem para a tela.
  } else {
    Serial.println("Camera nao iniciou");
  }
  bus = new Arduino_ESP32DSIPanel(
    20, 20, 40,                     // Pulso, recuo e avanço horizontal.
    4, 8, 20,                       // Pulso, recuo e avanço vertical.
    60000000);                      // Velocidade pedida do DSI.
  tft = new Arduino_DSI_Display(
    800, 1280, bus, rotationScreen, true,
    DISP_RST, jd9365_init_operations, sizeof(jd9365_init_operations) / sizeof(lcd_init_cmd_t)); //
    Tabela de início do chip JD9365.
  tft->begin();
  pinMode(23, OUTPUT);              // Luz de fundo no GPIO 23.
  digitalWrite(23, HIGH);
  myDisplay.setDrawObject(tft);
  myDisplay.setTouchCorners(0, 800, 0, 1280);
  myDisplay.setInvertAxis(false, false);
  myDisplay.setSwapAxis(false);
  myDisplay.startTouchGSL3680(DISPLAY_W, DISPLAY_H, rotationScreen, 7, 8, TOUCH_INT, TOUCH_RST); // SDA
    7, SCL 8.
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}
```

8.8 ESP32-P4, Guition 4 pol., GT911 e câmera OV02C10

`user_setup.h`:

```
#define DISP_DEFAULT
#define TOUCH_GT911
#define CAM_OV02C10
```

Copie o sketch da seção 10. Troque os números repetidos e troque o `setup` por este.

```
const int DISPLAY_W = 480;
```

```
const int DISPLAY_H = 800;
const int DISP_RST = 5;             // Nesta placa o reset da tela é o GPIO 5.
const int TOUCH_INT = -1;
const int TOUCH_RST = -1;

void setup() {
  Serial.begin(115200);
  myDisplay.setRotationCamera(0, false, false);
  if (myDisplay.startCamera(0, 8, 7, 400000, -1, -1, 1920, 1080)) {
    myDisplay.getCamera().startPpa();
  }
  bus = new Arduino_ESP32DSIPanel(
    12, 42, 42,                     // Tempos horizontais desta tela de 4 polegadas.
    2, 8, 166,                      // Tempos verticais.
    34000000);                      // Velocidade.
  tft = new Arduino_DSI_Display(
    480, 800, bus, 0, true,
    DISP_RST, st7701_init_operations, sizeof(st7701_init_operations) / sizeof(lcd_init_cmd_t)); // Chip
    ST7701.
  tft->begin();
  pinMode(23, OUTPUT);
  digitalWrite(23, HIGH);
  myDisplay.setDrawObject(tft);
  myDisplay.setTouchCorners(0, 480, 0, 800);
  myDisplay.setInvertAxis(false, false);
  myDisplay.setSwapAxis(false);
  myDisplay.startTouchGT911(480, 800, 0, 7, 8, TOUCH_INT, TOUCH_RST);
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}
```

8.9 ESP32-P4, placa Functional EV, EK79007, GT911 e câmera SC2336

Nesta placa o toque nasce invertido nos dois eixos, e a imagem da câmera nasce espelhada nos dois eixos. Copie o sketch da seção 10, troque os números repetidos e troque o `setup` por este.

`user_setup.h`:

```
#define DISP_DEFAULT
#define TOUCH_GT911
#define CAM_SC2336

const int DISPLAY_W = 1024;
const int DISPLAY_H = 600;
const bool TOUCH_INVERT_X = true;   // O dedo acompanha a tela com esta inversão.
const bool TOUCH_INVERT_Y = true;
const bool CAM_MIRROR_X = true;     // Espelha a imagem na horizontal.
const bool CAM_MIRROR_Y = true;     // Espelha a imagem na vertical.
const int DISP_RST = 27;
#define DISP_LED 26                  // Nesta placa a luz é o GPIO 26.

void setup() {
  Serial.begin(115200);
  myDisplay.setRotationCamera(0, CAM_MIRROR_X, CAM_MIRROR_Y);
  if (myDisplay.startCamera(0, 8, 7, 400000, -1, -1, 1920, 1080)) {
    myDisplay.getCamera().startPpa();
  }
  bus = new Arduino_ESP32DSIPanel(10, 160, 160, 1, 23, 12, 52000000); // Tempos do painel EK79007.
  tft = new Arduino_DSI_Display(
    1024, 600, bus, 0, true,
    DISP_RST, ek79007_init_operations, sizeof(ek79007_init_operations) / sizeof(lcd_init_cmd_t));
  tft->begin();
  pinMode(DISP_LED, OUTPUT);
  digitalWrite(DISP_LED, HIGH);
  myDisplay.setDrawObject(tft);
  myDisplay.setTouchCorners(0, 1024, 0, 600);
  myDisplay.setInvertAxis(TOUCH_INVERT_X, TOUCH_INVERT_Y);
  myDisplay.setSwapAxis(false);
  myDisplay.startTouchGT911(1024, 600, 0, 7, 8, -1, -1);
```

```
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}
```

8.10 ESP32-P4, Waveshare 7 pol. com câmera, ILI9881C e GT911 Waveshare

`user_setup.h`:

```
#define DISP_DEFAULT
#define TOUCH_GT911_WAVESHARE
#define CAM_OV5647
```

Copie o sketch da seção 10, troque os números repetidos e troque o `setup` por este.

```
const int DISPLAY_W = 720;
const int DISPLAY_H = 1280;
const int DISP_RST = -1;            // O reset deste painel fica no chip auxiliar.

void setup() {
  Serial.begin(115200);
  myDisplay.setRotationCamera(0, false, false);
  if (myDisplay.startCamera(0, 8, 7, 400000, -1, -1, 1920, 1080)) {
    myDisplay.getCamera().startPpa();
  }
  if (!myDisplay.waveshareDsiAuxBegin(ILI9881C_WAVESHARE_7_SDA, ILI9881C_WAVESHARE_7_SCL)) { // Liga o
    chip que controla luz e reset.
    Serial.println("Chip auxiliar do painel falhou");
  }
  bus = new Arduino_ESP32DSIPanel(
    ILI9881C_WAVESHARE_7_HSYNC_PULSE, ILI9881C_WAVESHARE_7_HSYNC_BACK, ILI9881C_WAVESHARE_7_HSYNC_FRONT,
    ILI9881C_WAVESHARE_7_VSYNC_PULSE, ILI9881C_WAVESHARE_7_VSYNC_BACK, ILI9881C_WAVESHARE_7_VSYNC_FRONT,
    ILI9881C_WAVESHARE_7_PIXEL_CLOCK_HZ, ILI9881C_WAVESHARE_7_LANE_MBPS); // Números que já vêm no
    arquivo do painel.
  tft = new Arduino_DSI_Display(
    720, 1280, bus, 0, true,
    DISP_RST, ili9881c_waveshare_7_init_operations, sizeof(ili9881c_waveshare_7_init_operations) /
    sizeof(lcd_init_cmd_t));
  tft->begin();
  myDisplay.waveshareDsiAuxBacklight(255); // 0 apaga a luz. 255 é o máximo.
  myDisplay.setDrawObject(tft);
  myDisplay.setTouchCorners(0, 720, 0, 1280);
  myDisplay.setInvertAxis(false, false);
  myDisplay.setSwapAxis(false);
  myDisplay.startTouchGT911Waveshare(720, 1280, 0, 7, 8); // Este toque só pede SDA e SCL.
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}
```

8.11 ESP32-P4-EYE, tela ST7789 e câmera OV2710, sem toque

`user_setup.h`:

```
#define DISP_DEFAULT
#define CAM_OV2710

#include <SPI.h>
#include <Arduino_GFX_Library.h>
#include <displaywx.h>
#include <ESP_Video.h>              // Driver de vídeo usado pela câmera.

const int DISPLAY_W = 240;
const int DISPLAY_H = 240;
const int DISP_FREQUENCY = 40000000;
const int CAM_SDA = 14;             // Nesta placa a câmera tem I2C próprio.
const int CAM_SCL = 13;
const int CAM_RST = 26;
const int CAM_PIN_CLK = 11;         // Clock que a câmera precisa para acordar.
```

```
const int CAM_PIN_PWR_EN = 12;       // Liga a energia da câmera e da tela.
const int DISP_RST = 15;
const int DISP_DC = 19;
const int DISP_CS = 18;
const int DISP_MOSI = 16;
const int DISP_MISO = -1;
const int DISP_SCLK = 17;
const int DISP_LED = 20;
const uint8_t rotationScreen = 0;
const bool isIPS = true;            // O exemplo desta placa usa IPS ligado.
Arduino_DataBus *bus = nullptr;
Arduino_GFX *tft = nullptr;
DisplayWX myDisplay;
void screen0();
void loadWidgets();

void setup() {
  Serial.begin(115200);
  if (!myDisplay.getCamera().enableBoardPower(CAM_PIN_PWR_EN)) { // Liga a alimentação. Sem isso a
    câmera não responde.
    Serial.println("Falha ao ligar a energia da camera");
    return;
  }
  if (!myDisplay.getCamera().startXclk(CAM_PIN_CLK, 24000000)) { // Entrega o clock de 24 MHz no GPIO
    11.
    return;
  }
  myDisplay.setRotationCamera(rotationScreen, false, false);
  if (myDisplay.startCamera(0, CAM_SCL, CAM_SDA, 400000, CAM_RST, -1, 1920, 1080)) { // A frequência I2C
     da câmera aqui é 400000. O 24000000 é só o clock do sensor.
    myDisplay.getCamera().startPpa();
  }
  pinMode(DISP_LED, OUTPUT);
  digitalWrite(DISP_LED, LOW);       // Nesta placa LOW acende a luz. Se ficar apagada, troque para
    HIGH.
  bus = new Arduino_ESP32SPIDMA(DISP_DC, DISP_CS, DISP_SCLK, DISP_MOSI, DISP_MISO, SPI2_HOST + 1,
    false); // SPI por DMA, sem objeto compartilhado, porque não há toque SPI.
  tft = new Arduino_ST7789(bus, DISP_RST, rotationScreen, isIPS, DISPLAY_W, DISPLAY_H, 0, 0, 0, 0); //
    Os quatro zeros finais são deslocamento e offset. Mantenha 0 nesta placa.
  tft->begin(DISP_FREQUENCY);
  myDisplay.setDrawObject(tft);
  loadWidgets();
  myDisplay.loadScreen(screen0);
  myDisplay.createTask(false, 3);
}

void loop() {
}

void screen0() {
  myDisplay.setBackgroundColor(CFK_BLACK); // Pinta o fundo de preto e guarda a cor. Troque CFK_BLACK
    por outra cor CFK_.
  myDisplay.drawWidgetsOnScreen(0);
}

void loadWidgets() {
}
```

9. Rotação, cores, IPS, espelho e SPI compartilhado

9.1 Rotação da tela

`rotationScreen` vale 0, 1, 2 ou 3.

Valor	Posição
0	Do jeito que o painel sai de fábrica
1	Um quarto de volta
2	Meia volta
3	Três quartos de volta

O mesmo número entra no construtor da tela e na função `startTouch`. Grave de novo depois de mudar.

Quando a tela gira e o dedo fica no lugar errado, ajuste o toque. Mude um valor de cada vez e teste um botão no canto.

• Se o movimento na horizontal mexe na vertical, mude `TOUCH_SWAP_XY` para `true`.

• Se a esquerda e a direita estão trocadas, mude `TOUCH_INVERT_X` para `true`.

• Se a parte de cima e a de baixo estão trocadas, mude `TOUCH_INVERT_Y` para `true`.

Exemplo real da placa ESP32-S3 4.3 pol. com GT911:

rotationScreen	TOUCH_SWAP_XY	TOUCH_INVERT_X	TOUCH_INVERT_Y
0	false	false	false
1	true	false	true
2	false	true	true
3	true	true	false

Na Guition 7 pol. com rotação 0, os três valores começam em `false`.

`setTouchCorners(x0, x1, y0, y1)` diz onde fica a área do dedo. Na maioria das placas, x0 e y0 são 0. x1 é a largura. y1 é a altura. Se o dedo só funciona numa faixa da tela, aumente ou diminua x1 e y1.

Para ver os números do dedo na serial, tire o comentário de `myDisplay.enableTouchLog()`.

9.2 BGR e RGB

A cor na memória tem a ordem vermelho, verde, azul. Alguns painéis leem azul, verde, vermelho.

Se o vermelho aparece azul, e o azul aparece vermelho, acrescente isto em `user_setup.h`:

```
#define IS_BGR // A biblioteca troca o vermelho com o azul nas cores CFK_.
```

Se o branco aparece preto, acrescente:

```
#define INVERT_COLORS_BITS // A biblioteca inverte os bits da cor.
```

Use as cores `CFK_RED`, `CFK_BLUE` e as outras `CFK_`. Elas passam por esse ajuste. Um número solto, como `0xF800`, não passa.

9.3 IPS

`isIPS` é um `true` ou `false` no sketch. Ele entra no construtor das telas SPI e de algumas telas RGB. O comentário gerado também chama essa flag de byte grande (big endian).

Se a imagem SPI fica com faixas ou cores quebradas, troque `isIPS` de `false` para `true`, ou o contrário, e grave de novo.

Nas telas DSI da Guition 7, o construtor `Arduino_DSI_Display` não recebe `isIPS`. O `true` que aparece nesse construtor pede atualização automática. Mudar `isIPS` nesse sketch DSI não muda a imagem.

9.4 Espelho do toque

O espelho do toque são as duas funções:

```
myDisplay.setInvertAxis(TOUCH_INVERT_X, TOUCH_INVERT_Y); // Primeiro valor espelha X. Segundo valor
    espelha Y.
myDisplay.setSwapAxis(TOUCH_SWAP_XY);                   // true troca o eixo X com o eixo Y.
```

Chame as duas antes de `startTouch`.

9.5 Espelho da câmera

```
myDisplay.setRotationCamera(rotationScreen, CAM_MIRROR_X, CAM_MIRROR_Y); // Rotação de 0 a 3, espelho
    horizontal, espelho vertical.
```

Chame isso antes de `startCamera`. Se o rosto aparece ao contrário, mude um dos dois `bool` para `true`.

`setCameraPreviewFps(numero)` aceita de 5 a 24. Esse número é a taxa da miniatura na tela.

9.6 SPI compartilhado entre toque e tela

Isso vale para o toque **XPT2046**. O GT911 e os outros toques capacitivos usam I2C. Eles não entram no SPI da tela.

Os fios compartilhados são CLK, MOSI e MISO. O CS da tela e o CS do toque são pinos diferentes. Só um CS fica ativo de cada vez.

Faça assim:

• Crie um `SPIClass spi_shared`.

• Chame `spi_shared.begin(CLK, MISO, MOSI)` uma vez.

• Entregue `&spi_shared` para `Arduino_HWSPI`.

• Entregue o mesmo `&spi_shared` para `startTouchXPT2046`, junto com o CS do toque e o CS da tela.

O exemplo completo está na seção 8.2.

No ESP32-P4 com tela DSI, a tela não usa SPI. O toque GT911 e a câmera dividem o I2C dos pinos 7 e 8. A biblioteca faz essa divisão. Deixe o `Wire.begin` desses pinos por conta dela.

9.7 Desenho feito por você

A tarefa da biblioteca desenha os widgets. Se você precisar desenhar um retângulo seu dentro de um callback, use o par:

```
myDisplay.startCustomDraw(); // Segura o desenho para ninguém pintar ao mesmo tempo.
```

```
tft->fillRect(10, 10, 40, 20, CFK_RED); // O seu desenho.
myDisplay.finishCustomDraw(); // Solta o desenho.
```

9.8 Fundo da tela

`setBackgroundColor` recebe um número de 16 bits. Nos exemplos, esse número é uma cor `CFK_`.

```
myDisplay.setBackgroundColor(CFK_WHITE); // Pinta o fundo e guarda a cor. Troque CFK_WHITE por CFK_BLACK
     ou outra cor CFK_.
```

A função faz as duas coisas ao mesmo tempo. Ela pinta a tela inteira. Ela guarda a mesma cor para os widgets apagarem o texto antigo.

Chame essa função depois de `myDisplay.setDrawObject(tft)`. Nos exemplos, a chamada fica dentro de `screen0`. Nesse momento a tela já foi entregue à biblioteca.

Se você chamar antes de `setDrawObject`, a cor fica só guardada. A tela ainda não é pintada.

10. Sketch base dos widgets

Todos os widgets da seção 12 em diante entram neste sketch. A placa é a ESP32-P4 com a Guition de 7 polegadas, toque GT911 e câmera OV5647.

10.1 user_setup.h

Apague o conteúdo antigo e cole isto:

```
#define DISP_DEFAULT // Tela colorida JD9165.
#define TOUCH_GT911  // Toque da Guition 7.
#define CAM_OV5647   // Câmera ligada no mesmo I2C do toque.
```

10.2 Sketch

Crie um sketch novo e cole o código. Os widgets das seções seguintes pedem para acrescentar linhas neste arquivo. O lugar de cada acréscimo está marcado com um comentário `ACRESCENTE`.

```
#include <SPI.h>                 // SPI fica incluído mesmo na tela DSI, porque outros módulos da IDE
    usam o cabeçalho.
#include <Arduino_GFX_Library.h> // Desenho e painel DSI.
#include <displaywx.h>           // Biblioteca de widgets.
#include <ESP_Video.h>           // Câmera MIPI.

const int DISPLAY_W = 1024;      // Largura da Guition 7. Mude só se trocar de painel.
const int DISPLAY_H = 600;       // Altura da Guition 7.
const int TOUCH_MAP_X0 = 0;      // Início do eixo X do dedo.
const int TOUCH_MAP_X1 = 1024;   // Fim do eixo X do dedo.
const int TOUCH_MAP_Y0 = 0;      // Início do eixo Y do dedo.
const int TOUCH_MAP_Y1 = 600;    // Fim do eixo Y do dedo.
const bool TOUCH_SWAP_XY = false;  // Troca X com Y. Veja a seção 9.
const bool TOUCH_INVERT_X = false; // Espelho horizontal do dedo.
const bool TOUCH_INVERT_Y = false; // Espelho vertical do dedo.
const int CAM_SDA = 7;           // Dado I2C da câmera. É o mesmo fio do toque.
const int CAM_SCL = 8;           // Relógio I2C da câmera. É o mesmo fio do toque.
const int CAM_FREQUENCY_SCCB = 400000; // Velocidade do I2C da câmera.
const int CAM_INT = -1;          // Esta ligação não usa interrupção da câmera.
const int CAM_RST = -1;          // Esta ligação não usa reset separado da câmera.
const int CAM_WIDTH = 1920;      // Largura pedida à OV5647. Pode ser 800, 1280 ou 1920, com a altura da
     tabela da seção 6.
const int CAM_HEIGHT = 1080;     // Altura pedida. O par 1920x1080 é o mais comum.
const int CAM_I2C_PORT = 0;      // Porta I2C número 0.
const bool CAM_MIRROR_X = false; // true espelha a imagem na horizontal.
```

```
const bool CAM_MIRROR_Y = false; // true espelha a imagem na vertical.
const int DISP_RST = 27;         // Reset do painel.
const int TOUCH_SCL = 8;         // Relógio do toque.
const int TOUCH_SDA = 7;         // Dado do toque.
const int TOUCH_INT = -1;        // Sem pino de interrupção nesta placa.
const int TOUCH_RST = 22;        // Reset do GT911.
const uint8_t rotationScreen = 0; // Rotação. 0, 1, 2 ou 3.
const bool isIPS = false;        // Nesta tela DSI o construtor não usa este valor. Ele fica aqui para
    você copiar o sketch para uma tela SPI.

const int PIN_BOTAO = 4;         // Botão em pulldown. Solto = 0. Apertado = 1.
const int PIN_LED = 5;           // LED digital.
const int PIN_POT = 6;           // Potenciômetro. Precisa ser um pino ADC.
const int PIN_PWM = 3;           // LED com brilho.

#define DISP_LED 23              // Luz de fundo da Guition 7.

Arduino_ESP32DSIPanel *bus = nullptr; // Barramento DSI. Nasce no setup.
Arduino_DSI_Display *tft = nullptr;   // Tela DSI. Nasce no setup.
DisplayWX myDisplay;                  // Objeto principal.

void screen0();                  // Pinta o fundo e os widgets.
void loadWidgets();              // Configura os widgets. Cada seção acrescenta código aqui.
// ACRESCENTE aqui o protótipo do callback do widget, quando o exemplo tiver um.

void setup() {
  Serial.begin(115200);          // Mensagens no monitor serial, 115200 bits por segundo.
  pinMode(PIN_BOTAO, INPUT);     // Entrada do botão. O pulldown é o resistor externo.
  pinMode(PIN_LED, OUTPUT);      // Saída do LED.
  digitalWrite(PIN_LED, LOW);    // LED começa apagado.
  pinMode(PIN_POT, INPUT);       // Entrada do potenciômetro.
  ledcAttach(PIN_PWM, 5000, 8);  // PWM de 5000 Hz, 8 bits. O brilho vai de 0 a 255.
  ledcWrite(PIN_PWM, 0);         // Brilho inicial zero.
  myDisplay.setRotationCamera(rotationScreen, CAM_MIRROR_X, CAM_MIRROR_Y); // Rotação e espelho da
    câmera, antes de iniciar.
  if (myDisplay.startCamera(CAM_I2C_PORT, CAM_SCL, CAM_SDA, CAM_FREQUENCY_SCCB, CAM_RST, CAM_INT,
    CAM_WIDTH, CAM_HEIGHT)) { // Liga a OV5647.
    myDisplay.getCamera().startPpa(); // Prepara a redução da imagem para caber no widget.
  } else {
    Serial.println("Camera nao iniciou"); // A tela continua. Confira o define CAM_OV5647 e os fios 7 e
    8.
  }
  bus = new Arduino_ESP32DSIPanel(
    40, 160, 160,                // Tempos horizontais do JD9165.
    10, 23, 12,                  // Tempos verticais do JD9165.
    48000000);                   // Velocidade pedida.
  tft = new Arduino_DSI_Display(
    1024, 600, bus, rotationScreen, true, // true pede atualização automática. Não é o flag IPS.
    DISP_RST, jd9165_init_operations, sizeof(jd9165_init_operations) / sizeof(lcd_init_cmd_t)); //
    Tabela de início do JD9165.
  tft->begin();                  // Liga o painel.
  pinMode(DISP_LED, OUTPUT);     // Luz de fundo.
  digitalWrite(DISP_LED, HIGH);  // HIGH acende a luz desta placa.
  myDisplay.setDrawObject(tft);  // Entrega a tela para a biblioteca.
  myDisplay.setTouchCorners(TOUCH_MAP_X0, TOUCH_MAP_X1, TOUCH_MAP_Y0, TOUCH_MAP_Y1); // Área do dedo.
  myDisplay.setInvertAxis(TOUCH_INVERT_X, TOUCH_INVERT_Y); // Espelho do dedo.
  myDisplay.setSwapAxis(TOUCH_SWAP_XY); // Troca de eixos do dedo.
  myDisplay.startTouchGT911(DISPLAY_W, DISPLAY_H, rotationScreen, TOUCH_SDA, TOUCH_SCL, TOUCH_INT,
    TOUCH_RST); // Inicia o GT911. A ordem é SDA, SCL, INT, RST.
  loadWidgets();                 // Monta os widgets que você acrescentou.
  myDisplay.loadScreen(screen0); // Mostra a tela 0.
  myDisplay.createTask(false, 3); // Começa a ler o toque e a desenhar.
}

void loop() {
  delay(30);                     // Cada exemplo de widget troca este loop.
}

void screen0() {
  myDisplay.setBackgroundColor(CFK_WHITE); // Pinta o fundo e guarda a cor para apagar texto. Troque
    CFK_WHITE por outra cor CFK_.
  myDisplay.drawWidgetsOnScreen(0); // Desenha os widgets da tela número 0.
```

```
}

void loadWidgets() {
  // ACRESCENTE aqui o setup do widget.
}
```

Como usar um exemplo de widget:

• Cole as variáveis globais do widget abaixo de `DisplayWX myDisplay`.

• Cole o protótipo do callback no lugar marcado, se o exemplo tiver callback.

• Cole o corpo do callback antes de `setup`.

• Troque `loadWidgets` pelo do exemplo.

• Troque `loop` pelo do exemplo.

O construtor do widget pede três números: posição X, posição Y e o número da tela. A tela deste guia é a 0. X e Y são o canto de cima à esquerda, em pixels. X cresce para a direita. Y cresce para baixo.

11. Funções que todo widget tem

A tarefa da tela chama sozinha `detectTouch`, `redraw`, `forceUpdate` e `getCallbackFunc`. O seu sketch chama as funções das seções seguintes.

Estas funções servem para qualquer widget:

Função	O que faz
`show()`	Mostra o widget.
`hide()`	Esconde o widget.
`setEnabled(true)`	O widget volta a aceitar toque.
`setEnabled(false)`	O widget para de aceitar toque.
`isEnabled()`	Diz se o toque está aceito.
`lock()`	Trava o toque.
`unlock()`	Destrava o toque.
`isLocked()`	Diz se o toque está travado.

Um callback é uma função sem argumentos e sem retorno:

```
void meuCallback() {            // A biblioteca chama esta função quando o widget é usado.
  digitalWrite(PIN_LED, HIGH);  // Exemplo: acende o LED.
}
```

O nome do callback entra no campo `callback` da configuração, sem parênteses: `callback: meuCallback`.

12. Label

O Label escreve texto. Ele não reage ao dedo.

Função	O que faz
`setup(config)`	Define texto, prefixo, sufixo, fonte, alinhamento e cores.
`setText("texto")`	Troca o texto.
`setText(String)`	Troca o texto a partir de um `String`.
`setTextInt(numero)`	Escreve um número inteiro.
`setTextFloat(numero, casas)`	Escreve um número com casas depois da vírgula.
`setPrefix("antes")`	Texto fixo antes do valor.
`setSuffix("depois")`	Texto fixo depois do valor.
`setDecimalPlaces(casas)`	Casas usadas por `setTextFloat`.
`setFontSize(tamanho)`	Tamanho da fonte simples. Com a fonte Roboto, prefira mudar `fontFamily`.
`show()` e `hide()`	Mostra ou esconde.

`datum` é o ponto de apoio do texto. `TL_DATUM` é o canto de cima à esquerda. `MC_DATUM` é o centro.

Cole abaixo de `DisplayWX myDisplay`:

```
Label rotulo(40, 40, 0);         // X, Y e tela 0.
const uint8_t qtdLabel = 1;      // Quantidade de labels.
Label *listaLabel[qtdLabel] = {&rotulo}; // A biblioteca recebe um vetor de ponteiros.
```

Troque `loadWidgets`:

```
void loadWidgets() {
  LabelConfig cfg = {};          // Zera a configuração.
  cfg.text = "Esperando";        // Texto inicial.
  cfg.prefix = "Estado: ";       // Fica antes do texto.
  cfg.suffix = "";               // Fica depois do texto. Vazio aqui.
  cfg.fontFamily = &RobotoRegular15pt7b; // Fonte. Troque o número 15 por 10, 20 ou 30.
  cfg.datum = TL_DATUM;          // O ponto X,Y é o canto de cima à esquerda do texto.
  cfg.fontColor = CFK_BLACK;     // Cor das letras.
  cfg.backgroundColor = CFK_WHITE; // Cor atrás das letras.
  rotulo.setup(cfg);             // Aplica a configuração.
  rotulo.setDecimalPlaces(0);    // Sem casas depois da vírgula neste exemplo.
  rotulo.setFontSize(1);         // Tamanho da fonte simples. A Roboto acima tem prioridade no desenho
    GFX.
  myDisplay.setLabel(listaLabel, qtdLabel); // Entrega o vetor para a biblioteca. Chame uma vez.
}
```

Troque `loop`:

```
void loop() {
  int leitura = analogRead(PIN_POT); // Lê o potenciômetro. O valor sobe quando você gira.
  bool apertado = digitalRead(PIN_BOTAO) == HIGH; // HIGH significa botão apertado, por causa do
    pulldown.
  digitalWrite(PIN_LED, apertado ? HIGH : LOW); // O LED segue o botão.
  if (apertado) {
    rotulo.setText("Apertado");  // Troca o miolo do texto. O prefixo continua.
```

```
  } else {
    rotulo.setTextInt(leitura);  // Mostra o número do potenciômetro.
  }
  int brilho = leitura >> 4;     // Reduz a leitura para caber em 0..255. Ajuste o deslocamento se o
    brilho não mudar.
  ledcWrite(PIN_PWM, brilho);    // Manda o brilho para o LED PWM.
  delay(50);                     // Pausa curta para a leitura assentar.
}
```

13. Led

O Led da tela é um círculo. Ele não muda quando o dedo toca. Você muda o estado pelo código. O LED de verdade no GPIO 5 segue esse estado.

Função	O que faz
`setup(config)`	Raio, cor ligada, cor desligada e estado inicial.
`setState(true ou false)`	Liga ou desliga o círculo.
`getState()`	Lê o estado.
`setColor(cor)`	Troca a cor de ligado.
`drawBackground()`	Desenha a base do círculo.
`show()` e `hide()`	Mostra ou esconde.

```
Led luzTela(80, 160, 0);         // Centro do círculo em X=80, Y=160, tela 0.
const uint8_t qtdLed = 1;
Led *listaLed[qtdLed] = {&luzTela};

void loadWidgets() {
  LedConfig cfg = {};
  cfg.radius = 28;               // Raio em pixels. Um valor bom fica entre 5 e 50.
  cfg.colorOn = CFK_LIME;        // Cor quando ligado.
  cfg.colorOff = CFK_GRAY;       // Cor quando desligado.
  cfg.initialState = false;      // Começa desligado.
  luzTela.setup(cfg);
  luzTela.setColor(CFK_GREEN);   // Troca a cor de ligado depois do setup, se você quiser.
  luzTela.setState(false);       // Estado explícito.
  myDisplay.setLed(listaLed, qtdLed);
}

void loop() {
  bool apertado = digitalRead(PIN_BOTAO) == HIGH; // O botão manda no círculo da tela.
  luzTela.setState(apertado);
  digitalWrite(PIN_LED, luzTela.getState() ? HIGH : LOW); // O LED do GPIO copia o círculo.
  delay(30);
}
```

14. CircleButton

Botão redondo. Cada toque alterna ligado e desligado. A cor interna usa `pressedColor` quando está ligado. `onClick()` simula esse toque físico neste botão na tela: o programa faz o que o dedo faria ali.

Função	O que faz
`setup(config)`	Callback, raio e cor do estado ligado.
`onClick()`	Simula o toque físico neste botão na tela. Inverte o estado e chama o callback.
`getStatus()`	`true` quando está ligado.
`setStatus(true ou false)`	Força o estado.
`show()` e `hide()`	Mostra ou esconde.

```
CircleButton botaoRedondo(200, 180, 0);
const uint8_t qtdCircle = 1;
CircleButton *listaCircle[qtdCircle] = {&botaoRedondo};

void aoTocarRedondo();            // Protótipo. Cole junto dos outros protótipos.

void aoTocarRedondo() {
  digitalWrite(PIN_LED, botaoRedondo.getStatus() ? HIGH : LOW); // O LED copia o botão da tela.
}

void loadWidgets() {
  CircleButtonConfig cfg = {};
  cfg.callback = aoTocarRedondo; // Nome da função, sem parênteses.
  cfg.radius = 40;               // A biblioteca limita o raio entre 5 e 200.
  cfg.pressedColor = CFK_ORANGE; // Cor interna no estado ligado.
  botaoRedondo.setup(cfg);
  botaoRedondo.setStatus(false); // Começa desligado.
  botaoRedondo.setEnabled(true); // Aceita toque.
  myDisplay.setCircleButton(listaCircle, qtdCircle);
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) { // O botão físico faz o mesmo que um toque na tela.
    botaoRedondo.onClick();      // Simula o dedo tocando este botão na tela.
    delay(200);                  // Espera o dedo sair, para não inverter várias vezes.
  }
}
```

15. RectButton

Botão retangular. Cada toque alterna o estado. `pressedColor` é a cor do estado ligado. `onClick()` simula esse toque físico neste botão na tela.

Função	O que faz
`setup(config)`	Callback, largura, altura e cor do estado ligado.
`onClick()`	Simula o toque físico neste botão na tela. Inverte o estado e chama o callback.
`changeState()`	Só inverte o estado. Não chama o callback.
`getStatus()`	Lê ligado ou desligado.
`setStatus(valor)`	Força o estado.

Função	O que faz
`getEnabled()`	Diz se o botão aceita toque.
`setEnabled(valor)`	Liga ou desliga a aceitação do toque.
`show()` e `hide()`	Mostra ou esconde.

```
RectButton botaoRet(80, 240, 0);
const uint8_t qtdRect = 1;
RectButton *listaRect[qtdRect] = {&botaoRet};
void aoTocarRet();

void aoTocarRet() {
  digitalWrite(PIN_LED, botaoRet.getStatus() ? HIGH : LOW);
}

void loadWidgets() {
  RectButtonConfig cfg = {};
  cfg.callback = aoTocarRet;
  cfg.width = 160;               // Largura em pixels.
  cfg.height = 60;               // Altura em pixels.
  cfg.pressedColor = CFK_BLUE;   // Cor quando o estado está ligado.
  botaoRet.setup(cfg);
  botaoRet.setEnabled(true);     // false faria o botão ignorar o dedo.
  botaoRet.setStatus(false);
  myDisplay.setRectButton(listaRect, qtdRect);
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    botaoRet.onClick();          // Simula o dedo tocando este botão na tela.
    delay(200);                  // Espera o dedo sair, para não inverter várias vezes.
  }
}
```

16. TextButton

Botão com texto. O toque chama o callback. `backgroundColor` é a cor normal do botão. Enquanto o dedo segura, a borda fica mais escura. `onClick()` simula esse toque físico neste botão na tela.

Função	O que faz
`setup(config)`	Texto, callback, fonte, tamanho, raio dos cantos e cores.
`onClick()`	Simula o toque físico neste botão na tela e chama o callback.
`getEnabled()`	Diz se aceita toque.
`setEnabled(valor)`	Liga ou desliga o toque.
`show()` e `hide()`	Mostra ou esconde.

```
TextButton botaoTexto(80, 320, 0);
const uint8_t qtdTexto = 1;
TextButton *listaTexto[qtdTexto] = {&botaoTexto};
void aoTocarTexto();
```

```
void aoTocarTexto() {
  digitalWrite(PIN_LED, !digitalRead(PIN_LED)); // Inverte o LED do GPIO.
}

void loadWidgets() {
  TextButtonConfig cfg = {};
  cfg.text = "Luz";              // Palavra dentro do botão. Mantenha o texto curto.
  cfg.callback = aoTocarTexto;
  cfg.fontFamily = &RobotoBold15pt7b; // Fonte em negrito.
  cfg.width = 180;
  cfg.height = 64;
  cfg.radius = 12;               // Cantos arredondados. 0 deixa o canto reto.
  cfg.backgroundColor = CFK_BLUE;
  cfg.textColor = CFK_WHITE;
  botaoTexto.setup(cfg);
  botaoTexto.setEnabled(true);
  myDisplay.setTextButton(listaTexto, qtdTexto);
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    botaoTexto.onClick();        // Simula o dedo tocando este botão na tela.
    delay(200);
  }
}
```

17. ToggleButton

Interruptor. Cada toque muda entre ligado e desligado. A bolinha desliza.

Função	O que faz
`setup(config)`	Callback, largura, altura e cor do estado ligado.
`changeState()`	Inverte o estado.
`getStatus()`	Lê o estado.
`setStatus(valor, disparaCallback)`	Força o estado. O segundo argumento `true` também chama o callback.
`show()` e `hide()`	Mostra ou esconde.

```
ToggleButton chave(80, 420, 0);
const uint8_t qtdChave = 1;
ToggleButton *listaChave[qtdChave] = {&chave};
void aoMudarChave();

void aoMudarChave() {
  digitalWrite(PIN_LED, chave.getStatus() ? HIGH : LOW);
}

void loadWidgets() {
  ToggleButtonConfig cfg = {};
  cfg.callback = aoMudarChave;
  cfg.width = 120;
  cfg.height = 48;
  cfg.pressedColor = CFK_GREEN;  // Cor da faixa quando a chave está ligada.
  chave.setup(cfg);
  chave.setStatus(false, false); // Estado desligado, sem chamar o callback na largada.
  myDisplay.setToggle(listaChave, qtdChave);
}
```

```
void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    chave.setStatus(true, true); // Liga a chave e chama o callback, que acende o LED.
    delay(200);
  }
}
```

18. CheckBox

Caixa de marcado. O toque alterna. `weight` é a espessura do visto: `CheckBoxWeight::LIGHT`, `MEDIUM` ou `HEAVY`.

Função	O que faz
`setup(config)`	Callback, espessura do visto, tamanho e cores.
`getStatus()`	`true` se está marcado.
`setStatus(valor)`	Marca ou desmarca.
`setSize(pixels)`	Muda o lado do quadrado.
`getSize()`	Lê o lado do quadrado.
`setColors(marcado, desmarcado)`	Troca as cores. O segundo 0 pede a cor automática.
`show()` e `hide()`	Mostra ou esconde.

```
CheckBox caixa(80, 500, 0);
const uint8_t qtdCaixa = 1;
CheckBox *listaCaixa[qtdCaixa] = {&caixa};
void aoMudarCaixa();

void aoMudarCaixa() {
  digitalWrite(PIN_LED, caixa.getStatus() ? HIGH : LOW);
}

void loadWidgets() {
  CheckBoxConfig cfg = {};
  cfg.callback = aoMudarCaixa;
  cfg.weight = CheckBoxWeight::HEAVY; // Visto grosso. Use LIGHT ou MEDIUM para um traço fino.
  cfg.size = 48;                 // Lado do quadrado.
  cfg.checkedColor = CFK_GREEN;
  cfg.uncheckedColor = CFK_WHITE;
  caixa.setup(cfg);
  caixa.setSize(56);             // Exemplo da função que muda o tamanho depois do setup.
  caixa.setColors(CFK_BLUE, CFK_GRAY); // Nova cor do marcado e do desmarcado.
  caixa.setStatus(false);
  myDisplay.setCheckbox(listaCaixa, qtdCaixa);
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    caixa.setStatus(!caixa.getStatus()); // O botão físico inverte a caixa.
    digitalWrite(PIN_LED, caixa.getStatus() ? HIGH : LOW);
    delay(200);
  }
}
```

19. RadioGroup

Grupo de opções. Só uma fica marcada. O construtor recebe apenas o número da tela. A posição de cada bolinha fica no vetor `radio_t`.

Função	O que faz
`setup(config)`	Lista de bolinhas, callback, raio, id do grupo, quantidade e id inicial.
`setSelected(id)`	Marca a bolinha com esse id.
`getSelected()`	Lê o id marcado.
`getGroupId()`	Lê o número do grupo.
`show()` e `hide()`	Mostra ou esconde o grupo.

```
RadioGroup grupo(0);             // Só o número da tela. As posições estão no vetor abaixo.
const uint8_t qtdGrupo = 1;
RadioGroup *listaGrupo[qtdGrupo] = {&grupo};
const radio_t opcoes[3] = {      // Três bolinhas. Mude x e y para espalhar.
  {80, 80, CFK_RED, 1},          // x, y, cor, id.
  {80, 150, CFK_GREEN, 2},
  {80, 220, CFK_BLUE, 3}
};
void aoMudarGrupo();

void aoMudarGrupo() {
  uint16_t id = grupo.getSelected(); // 1, 2 ou 3.
  if (id == 1) {
    ledcWrite(PIN_PWM, 40);      // Brilho baixo.
  } else if (id == 2) {
    ledcWrite(PIN_PWM, 120);     // Brilho médio.
  } else {
    ledcWrite(PIN_PWM, 255);     // Brilho máximo.
  }
}

void loadWidgets() {
  RadioGroupConfig cfg = {};
  cfg.buttons = opcoes;          // Vetor das bolinhas.
  cfg.callback = aoMudarGrupo;
  cfg.radius = 18;               // Raio de cada bolinha.
  cfg.group = 1;                 // Número deste grupo. Outro grupo na mesma tela usa outro número.
  cfg.amount = 3;                // Quantidade de bolinhas. Igual ao tamanho do vetor.
  cfg.defaultClickedId = 1;      // Id que começa marcado.
  grupo.setup(cfg);
  myDisplay.setRadioGroup(listaGrupo, qtdGrupo);
  Serial.println(grupo.getGroupId()); // Mostra o número do grupo na serial, para você conferir.
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    uint16_t proximo = grupo.getSelected() + 1; // Avança o id.
    if (proximo > 3) {
      proximo = 1;               // Volta para a primeira opção.
    }
    grupo.setSelected(proximo);  // Marca a próxima bolinha.
    aoMudarGrupo();              // Atualiza o PWM.
    delay(200);
  }
}
```

20. TouchArea

Área invisível. O dedo nessa área chama o callback. Sirva para um desenho seu que precisa de toque. `onClick()` simula esse toque físico nesta área na tela.

Função	O que faz
`setup(config)`	Callback, largura e altura.
`getStatus()`	`true` depois de um toque dentro da área.
`changeState()`	Inverte o estado interno.
`onClick()`	Simula o toque físico nesta área na tela e chama o callback.
`show()` e `hide()`	Liga ou desliga a área.

```
TouchArea area(400, 40, 0);      // Canto da área invisível.
const uint8_t qtdArea = 1;
TouchArea *listaArea[qtdArea] = {&area};
void aoTocarArea();

void aoTocarArea() {
  digitalWrite(PIN_LED, !digitalRead(PIN_LED));
}

void loadWidgets() {
  TouchAreaConfig cfg = {};
  cfg.callback = aoTocarArea;
  cfg.width = 200;               // Largura da área invisível.
  cfg.height = 120;              // Altura da área invisível.
  area.setup(cfg);
  myDisplay.setTouchArea(listaArea, qtdArea);
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    area.onClick();              // Simula o dedo tocando esta área na tela.
    delay(200);
  }
  if (area.getStatus()) {
    Serial.println("Area tocada"); // Confere o estado na serial.
  }
}
```

21. HSlider

Barra horizontal. O dedo arrasta o valor entre o mínimo e o máximo. O callback roda quando o valor muda.

Função	O que faz
`setup(config)`	Callback, label opcional, mínimo, máximo, raio, largura e cores.
`getValue()`	Lê o valor atual.
`setValue(numero)`	Move a barra para esse valor.

Função	O que faz
`drawBackground()`	Desenha a trilha.
`show()` e `hide()`	Mostra ou esconde.

`subtitle` pode ser `nullptr`. Se você passar um Label, a barra escreve o número nele. Esse Label também precisa de `setLabel`.

```
Label valorBarra(80, 40, 0);
const uint8_t qtdLabel = 1;
Label *listaLabel[qtdLabel] = {&valorBarra};
HSlider barra(80, 120, 0);
const uint8_t qtdBarra = 1;
HSlider *listaBarra[qtdBarra] = {&barra};
void aoMoverBarra();

void aoMoverBarra() {
  int valor = barra.getValue(); // 0 a 100 neste exemplo.
  ledcWrite(PIN_PWM, map(valor, 0, 100, 0, 255)); // Converte 0..100 para o brilho 0..255.
}

void loadWidgets() {
  LabelConfig texto = {};
  texto.text = "0";
  texto.prefix = "Valor ";
  texto.suffix = "";
  texto.fontFamily = &RobotoRegular15pt7b;
  texto.datum = TL_DATUM;
  texto.fontColor = CFK_BLACK;
  texto.backgroundColor = CFK_WHITE;
  valorBarra.setup(texto);
  myDisplay.setLabel(listaLabel, qtdLabel); // O Label precisa ser registrado além de servir de legenda.

  HSliderConfig cfg = {};
  cfg.callback = aoMoverBarra;
  cfg.subtitle = &valorBarra;    // A barra atualiza este Label. Use nullptr se não quiser texto.
  cfg.minValue = 0;
  cfg.maxValue = 100;
  cfg.radius = 16;               // Raio da bolinha que o dedo arrasta.
  cfg.width = 400;               // Comprimento da trilha.
  cfg.pressedColor = CFK_BLUE;
  cfg.backgroundColor = CFK_GRAY;
  barra.setup(cfg);
  barra.setValue(0);
  myDisplay.setHSlider(listaBarra, qtdBarra);
}

void loop() {
  int leitura = analogRead(PIN_POT);
  int valor = map(leitura, 0, 4095, 0, 100); // Se o máximo da sua leitura for outro, troque 4095.
  barra.setValue(valor);         // O potenciômetro move a barra.
  aoMoverBarra();                // Atualiza o LED de brilho.
  delay(40);
}
```

22. SpinBox

Caixa com menos e mais. Cada toque soma ou tira `step`, dentro do mínimo e do máximo.

Função	O que faz
`setup(config)`	Callback, mínimo, máximo, valor inicial, tamanho, passo e cores.
`getValue()`	Lê o número.
`setValue(numero)`	Escreve o número, ainda dentro do mínimo e do máximo.
`drawBackground()`	Desenha a caixa e os botões.
`show()` e `hide()`	Mostra ou esconde.

```
SpinBox giro(80, 200, 0);
const uint8_t qtdGiro = 1;
SpinBox *listaGiro[qtdGiro] = {&giro};
void aoGirar();

void aoGirar() {
  ledcWrite(PIN_PWM, constrain(giro.getValue(), 0, 255)); // O valor vira brilho. constrain segura entre
     0 e 255.
}

void loadWidgets() {
  SpinBoxConfig cfg = {};
  cfg.callback = aoGirar;
  cfg.minValue = 0;
  cfg.maxValue = 255;
  cfg.startValue = 0;
  cfg.width = 220;
  cfg.height = 60;
  cfg.step = 15;                 // Quanto sobe ou desce a cada toque.
  cfg.color = CFK_BLUE;          // Cor dos botões.
  cfg.textColor = CFK_BLACK;
  giro.setup(cfg);
  giro.setValue(30);             // Valor inicial diferente do startValue, para mostrar a função.
  myDisplay.setSpinbox(listaGiro, qtdGiro);
}

void loop() {
  int leitura = analogRead(PIN_POT);
  giro.setValue(map(leitura, 0, 4095, 0, 255));
  aoGirar();
  delay(40);
}
```

23. NumberBox

Caixa numérica. O toque abre o teclado numérico. Ao terminar, a biblioteca chama de novo a função da tela, que neste guia é `screen0`. `onClick()` simula esse toque físico nesta caixa na tela.

Função	O que faz
`setup(config)`	Tela pai, callback, fonte, valor inicial, tamanho, cores e casas.
`onClick()`	Simula o toque físico nesta caixa na tela e abre o teclado numérico.
`setValue(numero)`	Escreve um número. O tipo é `double`.
`getValue()`	Lê o número.

Função	O que faz
`getValueChar()`	Lê o texto do número.
`convertoToString(numero, buffer, tamanho)`	Converte um número para texto no buffer. O nome da função é este mesmo, com o erro de grafia.
`getConfig()`	Devolve uma cópia da configuração.
`show()` e `hide()`	Mostra ou esconde.

`setNumberbox` prepara o teclado. Você não chama `setupNumpad` no sketch.

```
NumberBox numero(80, 300, 0);
const uint8_t qtdNumero = 1;
NumberBox *listaNumero[qtdNumero] = {&numero};
void aoDigitarNumero();

void aoDigitarNumero() {
  double valor = numero.getValue(); // Número que a pessoa digitou.
  ledcWrite(PIN_PWM, constrain((int)valor, 0, 255));
  Serial.println(numero.getValueChar()); // O mesmo número em texto.
}

void loadWidgets() {
  NumberBoxConfig cfg = {};
  cfg.funcPtr = screen0;         // Função que redesenha a tela quando o teclado fecha.
  cfg.callback = aoDigitarNumero;
  cfg.font = &RobotoRegular20pt7b;
  cfg.startValue = 10;           // Número inicial.
  cfg.width = 220;
  cfg.height = 60;
  cfg.letterColor = CFK_BLACK;
  cfg.backgroundColor = CFK_WHITE;
  cfg.decimalPlaces = 1;         // Uma casa depois da vírgula. Use 0 para inteiro.
  numero.setup(cfg);
  numero.setValue(12.5);         // Substitui o valor inicial.
  char texto[16];                // Buffer para a conversão.
  numero.convertoToString(numero.getValue(), texto, sizeof(texto)); // Enche o buffer.
  Serial.println(texto);
  myDisplay.setNumberbox(listaNumero, qtdNumero); // Esta chamada também prepara o teclado.
  NumberBoxConfig copia = numero.getConfig(); // Cópia para você ler as casas decimais.
  Serial.println(copia.decimalPlaces);
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    numero.onClick();            // Simula o dedo tocando esta caixa na tela. aoDigitarNumero roda só na
     confirmação.
    delay(200);
  }
}
```

24. TextBox

Caixa de texto. O toque abre o teclado de letras. Ao terminar, a tela `screen0` é chamada de novo. `onClick()` simula esse toque físico nesta caixa na tela.

Função	O que faz
`setup(config)`	Texto inicial, tela pai, callback, fonte, tamanho e cores.

Função	O que faz
`onClick()`	Simula o toque físico nesta caixa na tela e abre o teclado de letras.
`setValue("texto")`	Troca o texto.
`getValue()`	Lê o texto.
`show()` e `hide()`	Mostra ou esconde.

`setTextbox` prepara o teclado. O campo do callback nesta configuração se chama `cb`.

```
TextBox caixaTexto(80, 400, 0);
const uint8_t qtdCaixaTexto = 1;
TextBox *listaCaixaTexto[qtdCaixaTexto] = {&caixaTexto};
void aoDigitarTexto();

void aoDigitarTexto() {
  Serial.println(caixaTexto.getValue()); // Mostra o texto digitado.
  digitalWrite(PIN_LED, HIGH);   // Acende o LED para avisar que o texto mudou.
  delay(100);
  digitalWrite(PIN_LED, LOW);
}

void loadWidgets() {
  TextBoxConfig cfg = {};
  cfg.startValue = "Ola";        // Texto inicial.
  cfg.funcPtr = screen0;         // Redesenha a tela quando o teclado fecha.
  cfg.cb = aoDigitarTexto;       // Neste widget o campo se chama cb.
  cfg.font = &RobotoRegular15pt7b;
  cfg.width = 320;
  cfg.height = 60;
  cfg.letterColor = CFK_BLACK;
  cfg.backgroundColor = CFK_WHITE;
  caixaTexto.setup(cfg);
  caixaTexto.setValue("Sala 1"); // Troca o texto inicial.
  myDisplay.setTextbox(listaCaixaTexto, qtdCaixaTexto);
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    caixaTexto.onClick();        // Simula o dedo tocando esta caixa na tela. aoDigitarTexto roda só na
    confirmação.
    delay(200);
  }
}
```

25. VBar

Barra que enche conforme o valor. `orientation` pode ser `Orientation::VERTICAL` ou `Orientation::HORIZONTAL`.

Função	O que faz
`setup(config)`	Label opcional, mínimo, máximo, canto, orientação, tamanho e cor.
`setValue(numero)`	Enche a barra até esse valor.
`setMinValue` e `setMaxValue`	Mudam um limite.

Função	O que faz
`getMinValue` e `getMaxValue`	Leem os limites.
`setScale(minimo, maximo)`	Troca os dois limites juntos.
`drawBackground()`	Desenha a moldura.
`show()` e `hide()`	Mostra ou esconde.

```
VBar coluna(500, 40, 0);
const uint8_t qtdColuna = 1;
VBar *listaColuna[qtdColuna] = {&coluna};

void loadWidgets() {
  VerticalBarConfig cfg = {};
  cfg.subtitle = nullptr;        // Sem Label. Para mostrar o número, crie um Label e coloque o ponteiro
     aqui, e também registre esse Label.
  cfg.minValue = 0;
  cfg.maxValue = 100;
  cfg.round = 8;                 // Raio do canto. 0 deixa reto.
  cfg.orientation = Orientation::VERTICAL; // Use HORIZONTAL para deitar a barra.
  cfg.width = 40;
  cfg.height = 240;
  cfg.filledColor = CFK_BLUE;
  coluna.setup(cfg);
  coluna.setScale(0, 255);       // Nova escala, para casar com o brilho.
  coluna.setMinValue(0);         // As duas funções separadas fazem o mesmo que setScale.
  coluna.setMaxValue(255);
  coluna.setValue(0);
  myDisplay.setVBar(listaColuna, qtdColuna);
  Serial.println(coluna.getMinValue());
  Serial.println(coluna.getMaxValue());
}

void loop() {
  int leitura = analogRead(PIN_POT);
  int valor = map(leitura, 0, 4095, coluna.getMinValue(), coluna.getMaxValue());
  coluna.setValue(valor);
  ledcWrite(PIN_PWM, valor);     // A barra e o LED PWM mostram o mesmo número.
  delay(40);
}
```

26. VAnalog

Escala com uma seta. O segundo argumento de `setValue` pede para desenhar o número.

Função	O que faz
`setup(config)`	Mínimo, máximo, tamanho, cores e quantidade de marcas.
`setValue(numero, mostrarNumero)`	Move a seta. `true` mostra o número.
`setMinValue`, `setMaxValue`, `getMinValue`, `getMaxValue`, `setScale`	Escala, como na VBar.
`drawBackground()`	Desenha a escala.
`show()` e `hide()`	Mostra ou esconde.

```
VAnalog escala(620, 40, 0);
const uint8_t qtdEscala = 1;
VAnalog *listaEscala[qtdEscala] = {&escala};

void loadWidgets() {
  VerticalAnalogConfig cfg = {};
  cfg.minValue = 0;
  cfg.maxValue = 100;
  cfg.width = 140;
  cfg.height = 280;
  cfg.arrowColor = CFK_RED;
  cfg.textColor = CFK_BLACK;
  cfg.backgroundColor = CFK_WHITE;
  cfg.borderColor = CFK_BLACK;
  cfg.steps = 5;                // Quantidade de divisões da escala.
  escala.setup(cfg);
  escala.setScale(0, 255);
  escala.setValue(0, true);     // true pede o número na tela.
  myDisplay.setVAnalog(listaEscala, qtdEscala);
}

void loop() {
  int leitura = analogRead(PIN_POT);
  int valor = map(leitura, 0, 4095, escala.getMinValue(), escala.getMaxValue());
  escala.setValue(valor, true);
  ledcWrite(PIN_PWM, valor);
  delay(40);
}
```

27. CircularBar

Arco que enche. Os ângulos vão de 0 a 360. `inverted` true enche no sentido contrário.

Função	O que faz
`setup(config)`	Mínimo, máximo, raio, ângulos, cores, espessura, número visível e sentido.
`setValue(numero)`	Enche o arco.
`setMinValue`, `setMaxValue`, `getMinValue`, `getMaxValue`, `setScale`	Escala.
`drawBackground()`	Desenha o arco vazio.
`show()` e `hide()`	Mostra ou esconde.

```
CircularBar arco(860, 180, 0);   // O ponto é o centro do arco.
const uint8_t qtdArco = 1;
CircularBar *listaArco[qtdArco] = {&arco};

void loadWidgets() {
  CircularBarConfig cfg = {};
  cfg.minValue = 0;
  cfg.maxValue = 100;
  cfg.radius = 80;
  cfg.startAngle = 135;         // Onde o arco começa, em graus.
  cfg.endAngle = 405;           // Onde o arco termina. Pode passar de 360.
  cfg.color = CFK_GREEN;        // Cor da parte cheia.
  cfg.backgroundColor = CFK_GRAY;
  cfg.textColor = CFK_BLACK;
```

```
  cfg.backgroundText = CFK_WHITE;
  cfg.thickness = 12;            // Espessura do traço.
  cfg.showValue = true;          // Mostra o número no centro.
  cfg.inverted = false;          // true inverte o sentido do enchimento.
  arco.setup(cfg);
  arco.setScale(0, 255);
  arco.setValue(0);
  myDisplay.setCircularBar(listaArco, qtdArco);
}

void loop() {
  int leitura = analogRead(PIN_POT);
  int valor = map(leitura, 0, 4095, arco.getMinValue(), arco.getMaxValue());
  arco.setValue(valor);
  ledcWrite(PIN_PWM, valor);
  delay(40);
}
```

28. Gauge

Medidor com agulha. As faixas coloridas usam dois vetores do mesmo tamanho: valores e cores. O máximo de faixas é 10. O título aceita até 20 letras.

Função	O que faz
`setup(config)`	Título, faixas, fonte, escala, tamanho, cores e marcas.
`setValue(numero)`	Move a agulha.
`drawBackground()`	Desenha o arco e as faixas.
`show()` e `hide()`	Mostra ou esconde.

```
GaugeSuper medidor(80, 40, 0);
const uint8_t qtdMedidor = 1;
GaugeSuper *listaMedidor[qtdMedidor] = {&medidor};
const int faixas[] = {80, 160, 255}; // Limites das cores. Ficam entre o mínimo e o máximo.
const uint16_t coresFaixa[] = {CFK_GREEN, CFK_YELLOW, CFK_RED};

void loadWidgets() {
  GaugeConfig cfg = {};
  cfg.title = "Nivel";           // Título. Use nullptr para omitir.
  cfg.intervals = faixas;
  cfg.colors = coresFaixa;
  cfg.fontFamily = &RobotoRegular10pt7b;
  cfg.minValue = 0;
  cfg.maxValue = 255;
  cfg.width = 280;
  cfg.height = 200;
  cfg.borderColor = CFK_BLACK;
  cfg.textColor = CFK_BLACK;
  cfg.backgroundColor = CFK_WHITE;
  cfg.titleColor = CFK_BLUE;
  cfg.needleColor = CFK_RED;     // Cor da agulha.
  cfg.markersColor = CFK_BLACK;  // Cor dos traços.
  cfg.amountIntervals = 3;       // Igual à quantidade de itens nos dois vetores.
  cfg.showLabels = true;         // Escreve os números das faixas. Peça uma fonte quando isto for true.
  medidor.setup(cfg);
  medidor.setValue(0);
  myDisplay.setGauge(listaMedidor, qtdMedidor);
}

void loop() {
```

```
  int leitura = analogRead(PIN_POT);
  int valor = map(leitura, 0, 4095, 0, 255);
  medidor.setValue(valor);
  ledcWrite(PIN_PWM, valor);
  delay(40);
}
```

29. Thermometer

Termômetro. O valor é `float`. `unit` é o texto da unidade, por exemplo `"C"`.

Função	O que faz
`setup(config)`	Label opcional, unidade, escala, tamanho, cores e casas.
`setValue(numero)`	Move a coluna de líquido.
`setMinValue`, `setMaxValue`, `getMinValue`, `getMaxValue`, `setScale`	Escala.
`drawBackground()`	Desenha o vidro e as marcas.
`show()` e `hide()`	Mostra ou esconde.

```
Thermometer termo(900, 40, 0);
const uint8_t qtdTermo = 1;
Thermometer *listaTermo[qtdTermo] = {&termo};

void loadWidgets() {
  ThermometerConfig cfg = {};
  cfg.subtitle = nullptr;        // Opcional. Um Label aqui mostra o número, e esse Label também entra
    em setLabel.
  cfg.unit = "C";                // Texto da unidade.
  cfg.minValue = 0;
  cfg.maxValue = 100;
  cfg.width = 70;
  cfg.height = 260;
  cfg.filledColor = CFK_RED;
  cfg.backgroundColor = CFK_WHITE;
  cfg.markColor = CFK_BLACK;
  cfg.decimalPlaces = 0;         // Casas do número, se houver Label.
  termo.setup(cfg);
  termo.setScale(0, 255);
  termo.setValue(0);
  myDisplay.setThermometer(listaTermo, qtdTermo);
}

void loop() {
  int leitura = analogRead(PIN_POT);
  float valor = map(leitura, 0, 4095, termo.getMinValue(), termo.getMaxValue());
  termo.setValue(valor);
  ledcWrite(PIN_PWM, (int)valor);
  delay(40);
}
```

30. LineChart

Gráfico de linhas. `push` acrescenta um ponto. A escala vertical deste exemplo vem de `minValue` e `maxValue`. A constante `LineChart::SHOW_ALL` vale 9999 e está disponível na classe.

Dá para ter até 10 séries. `workInBackground` true permite chamar `push` a partir do `loop`.

Função	O que faz
`setup(config)`	Cores, legendas, fonte, escala, tamanho, grade e estilo da linha.
`push(indiceDaSerie, valor)`	Acrescenta um ponto. Devolve true quando o ponto entrou.
`drawBackground()`	Desenha a grade.
`show()` e `hide()`	Mostra ou esconde.

```
LineChart grafico(40, 40, 0);
const uint8_t qtdGrafico = 1;
LineChart *listaGrafico[qtdGrafico] = {&grafico};
uint16_t corSerie[1] = {CFK_BLUE}; // Uma cor por série.

void loadWidgets() {
  LineChartConfig cfg = {};
  cfg.colorsSeries = corSerie;
  cfg.subtitles = nullptr;       // Sem legenda. Um vetor de Label* escreve o último ponto.
  cfg.font = &RobotoRegular10pt7b;
  cfg.minValue = 0;
  cfg.maxValue = 255;
  cfg.width = 500;
  cfg.height = 240;
  cfg.gridColor = CFK_GRAY;
  cfg.borderColor = CFK_BLACK;
  cfg.backgroundColor = CFK_WHITE;
  cfg.textColor = CFK_BLACK;
  cfg.verticalDivision = 4;      // Linhas horizontais da grade.
  cfg.maxPointsAmount = 40;      // Quantos pontos a linha guarda.
  cfg.amountSeries = 1;          // Uma linha só.
  cfg.workInBackground = true;   // Permite push dentro do loop.
  cfg.showZeroLine = true;       // Desenha a linha do zero.
  cfg.boldLine = true;           // Linha grossa.
  cfg.showDots = false;          // true desenha um ponto em cada amostra.
  grafico.setup(cfg);
  myDisplay.setLineChart(listaGrafico, qtdGrafico);
}

void loop() {
  int leitura = analogRead(PIN_POT);
  int valor = map(leitura, 0, 4095, 0, 255);
  grafico.push(0, valor);        // Série 0. A segunda série seria o índice 1.
  ledcWrite(PIN_PWM, valor);
  delay(100);                    // Um ponto a cada 100 ms. Diminua para um gráfico mais rápido.
}
```

31. Image

Imagem. Há dois inícios: pixels dentro do programa, ou arquivo no cartão, no SPIFFS ou no FAT.

Função	O que faz
`setupFromPixels(config)`	Usa um vetor de pixels.
`setupFromFile(config)`	Usa um arquivo. `source` pode ser `SourceFile::SD`, `SPIFFS` ou `FATFS`.
`setDrawBackground(true ou false)`	true pinta o fundo atrás da imagem.
`drawBackground()`	Desenha esse fundo.
`draw()`	Desenha a imagem na hora.
`getMetrics()`	Lê tempos de desenho.
`resetMetrics()`	Zera esses tempos.
`printMetrics()`	Escreve os tempos na serial.
`show()` e `hide()`	Mostra ou esconde.

O exemplo abaixo é uma imagem de 4 por 4 pixels, desenhada maior pela repetição visual dos pixels. Toque na imagem para acender o LED.

```
const uint16_t pixelsDemo[16] = { // 4 colunas por 4 linhas. A ordem é linha por linha.
  CFK_RED, CFK_RED, CFK_GREEN, CFK_GREEN,
  CFK_RED, CFK_RED, CFK_GREEN, CFK_GREEN,
  CFK_BLUE, CFK_BLUE, CFK_YELLOW, CFK_YELLOW,
  CFK_BLUE, CFK_BLUE, CFK_YELLOW, CFK_YELLOW
};
Image figura(40, 40, 0);
const uint8_t qtdFigura = 1;
Image *listaFigura[qtdFigura] = {&figura};
void aoTocarFigura();

void aoTocarFigura() {
  digitalWrite(PIN_LED, HIGH);
  delay(150);
  digitalWrite(PIN_LED, LOW);
}

void loadWidgets() {
  ImageFromPixelsConfig cfg = {};
  cfg.pixels = pixelsDemo;       // Vetor. Para uma foto grande, use setupFromFile.
  cfg.maskAlpha = nullptr;       // Sem máscara. Um vetor de bytes faz transparência.
  cfg.cb = aoTocarFigura;
  cfg.angle = 0;                 // Graus. 90 gira um quarto de volta.
  cfg.width = 4;
  cfg.height = 4;
  cfg.backgroundColor = CFK_WHITE;
  figura.setupFromPixels(cfg);
  figura.setDrawBackground(true);
  figura.draw();                 // Desenha agora. A tarefa também redesenha depois.
  figura.resetMetrics();         // Zera o contador de tempo.
  myDisplay.setImage(listaFigura, qtdFigura);
  figura.printMetrics();         // Primeira leitura, ainda quase vazia.
}

void loop() {
  if (digitalRead(PIN_BOTAO) == HIGH) {
    figura.draw();               // O botão pede um novo desenho.
    const PerformanceMetrics_t &tempos = figura.getMetrics(); // Referência aos contadores.
    Serial.println(tempos.drawCount); // Quantas vezes draw rodou.
    delay(200);
  }
```

```
}
```

Para carregar um arquivo, no lugar de `setupFromPixels`:

```
void loadWidgets() {
  ImageFromFileConfig arquivo = {};
  arquivo.path = "/foto.bmp";    // Caminho no sistema de arquivos. Troque pelo seu arquivo.
  arquivo.cb = aoTocarFigura;
  arquivo.source = SourceFile::SPIFFS; // Ou SourceFile::SD, ou SourceFile::FATFS.
  arquivo.backgroundColor = CFK_WHITE;
  figura.setupFromFile(arquivo);
  myDisplay.setImage(listaFigura, qtdFigura);
}
```

O cartão SD, quando você for usar, inicia com `myDisplay.startSD(pinoCS, &spi_shared, velocidade)`. Na Guition 7 a tela é DSI, então o SPI do cartão é um SPI à parte. Escolha pinos livres.

32. CameraMipi

Retângulo com a imagem da câmera. Dois toques seguidos abrem a imagem na tela inteira. Outro par de toques volta ao retângulo. `onClick()` simula esse toque físico neste retângulo na tela.

A câmera já foi ligada no sketch base. Esta seção só cria o retângulo.

Função que você chama	O que faz
`setup(config)`	Largura, altura, texto do zoom, texto do fps e quadros por segundo de 5 a 24.
`onClick()`	Simula o toque físico neste retângulo na tela. Abre a tela inteira. Se ele já estiver em tela inteira, volta ao tamanho normal.
`setZoomLevel(nivel)`	Zoom. O menor valor aceito é 1.
`getZoomLevel()`	Lê o zoom pedido.
`zoomLabel()`	Texto curto do zoom, para a tela.
`setCenterPoint(x, y)`	Ponto da imagem que fica no centro ao dar zoom.
`centerPointX()` e `centerPointY()`	Leem esse ponto.
`setKeepZoomOnMaximized(true)`	Mantém o zoom quando a imagem vai para a tela inteira.
`keepZoomOnMaximized()`	Lê essa escolha.
`leaveFullScreen()`	Sai da tela inteira neste widget.
`isFullScreen()`	Diz se este widget está em tela inteira.
`hasFullScreen()`	Diz se algum widget de câmera está em tela inteira.
`exitFullScreen()`	Limpa a marca global de tela inteira.
`openConfig()`	Abre o ajuste da câmera. Devolve true se abriu.
`getXPos`, `getYPos`, `getWidth`, `getHeight`	Posição e tamanho do retângulo.

Função que você chama	O que faz
`showZoomLevel`, `showFps`, `fps`	Leem o que foi pedido na configuração.
`setEnabled` e `getEnabled`	Ligam ou desligam o widget.
`getStatus` e `setStatus`	Estado interno ligado ou desligado.
`changeState`	Inverte esse estado.
`drawBackground`	Desenha a moldura.
`show` e `hide`	Mostra ou esconde.

`getXPlot`, `getYPlot`, `getWidthPlot`, `getHeightPlot`, `isRedrawDue`, `noteRedraw` e `noteZoomClamped` são usados pela tarefa que pinta o quadro. O sketch de exemplo não precisa chamá-las.

```
CameraMipi cameraTela(40, 40, 0); // Canto do retângulo da imagem.
const uint8_t qtdCamera = 1;
CameraMipi *listaCamera[qtdCamera] = {&cameraTela};
uint8_t nivelZoom = 1;           // Zoom atual, controlado pelo botão.

void loadWidgets() {
  CameraMipiConfig cfg = {};
  cfg.width = 320;               // Largura do retângulo na tela, em pixels.
  cfg.height = 240;              // Altura do retângulo.
  cfg.showZoomLevel = true;      // Escreve o zoom no canto.
  cfg.showFps = true;            // Escreve os quadros por segundo.
  cfg.fps = 8;                   // De 5 a 24. Suba se a imagem ficar lenta e a placa aguentar.
  cameraTela.setup(cfg);
  cameraTela.setZoomLevel(1);    // Zoom 1 mostra a imagem inteira.
  cameraTela.setCenterPoint(160, 120); // Centro do recorte quando o zoom for maior que 1.
  cameraTela.setKeepZoomOnMaximized(true); // O zoom continua na tela inteira.
  cameraTela.setEnabled(true);
  cameraTela.setStatus(true);
  myDisplay.setCameraMipi(listaCamera, qtdCamera);
  myDisplay.setCameraPreviewFps(cfg.fps); // Avisa a tarefa da câmera sobre a taxa.
  Serial.println(cameraTela.zoomLabel());
  Serial.println(cameraTela.getWidth());
  Serial.println(cameraTela.getHeight());
}

void loop() {
  int leitura = analogRead(PIN_POT);
  ledcWrite(PIN_PWM, map(leitura, 0, 4095, 0, 255)); // O potenciômetro manda no brilho, independente da
     câmera.
  if (digitalRead(PIN_BOTAO) == HIGH) {
    nivelZoom = nivelZoom + 1;   // 1, depois 2, depois 4, depois volta a 1.
    if (nivelZoom > 4) {
      nivelZoom = 1;
    }
    cameraTela.setZoomLevel(nivelZoom);
    Serial.println(cameraTela.getZoomLevel());
    if (nivelZoom == 1 && cameraTela.isFullScreen()) {
      cameraTela.leaveFullScreen(); // No zoom 1, se estiver em tela cheia, volta ao retângulo.
    }
    delay(200);
  }
}
```

Dois toques rápidos no retângulo abrem a tela inteira. `cameraTela.onClick()` simula esse toque físico no retângulo: a primeira chamada abre a tela inteira e a próxima, com este retângulo já aberto, volta ao tamanho normal. `CameraMipi::hasFullScreen()` diz se isso aconteceu. `CameraMipi::exitFullScreen()` limpa essa marca global.

Se `openConfig()` devolver false, a câmera não está pronta. Confira `CAM_OV5647` em `user_setup.h` e a mensagem `Camera nao iniciou` na serial.

Como mudar um exemplo com segurança

• Mude um número por vez.

• Grave.

• Olhe a tela.

• Se o dedo ficar torto, volte na seção 9 e mude um flag do toque.

• Se a cor ficar trocada, acrescente `IS_BGR` ou `INVERT_COLORS_BITS` em `user_setup.h`.

• Se a câmera não aparecer, confira se o chip da placa é ESP32-P4 e se o define da câmera é o do sensor soldado.

Guia do DMA da câmera

Este texto explica o DMA usado nos exemplos de câmera do ESP32-P4. A linguagem é simples. Cada linha de código tem um comentário.

Os exemplos ficam em `examples/generic/ESP32P4/cameras/`. Todos chamam `startPpa()`. Esse nome liga o bloco de hardware que carrega a foto da câmera até a memória da tela.

1. O que é DMA

DMA quer dizer acesso direto à memória.

Pense numa pilha de caixas. Sem DMA, o processador pega uma caixa, anda até o outro lado e solta. Depois volta e pega a próxima. Uma foto tem mais de dois milhões de caixas. O processador ficaria ocupado só com isso.

Com DMA, outro funcionário da placa carrega a pilha inteira. O processador só diz: a foto está aqui, a tela está ali, pode levar.

Nos exemplos, esse funcionário é o PPA do ESP32-P4. PPA quer dizer acelerador de pixels. A função `startPpa()` contrata esse funcionário. Sem ela, a foto é capturada e não chega na tela por esse caminho.

2. Por que a câmera precisa disso

Uma foto de 1920 por 1080 tem 2.073.600 pontos. Cada ponto é uma cor de 2 bytes. Uma foto ocupa cerca de 4 megabytes.

O retângulo na tela é bem menor. No exemplo da Guition de 7 polegadas ele tem 232 por 184 pontos. Alguém precisa escolher um pedaço da foto grande, diminuir e colocar nesse retângulo.

Se o processador fizesse isso ponto por ponto, a tela ficaria lenta e o resto do programa esperaria. O DMA faz o transporte. O processador só marca o começo e o fim.

3. O caminho da foto

A foto ao vivo passa por três lugares. O processador não copia os pontos no meio do caminho.

```
câmera  ->  memória da foto  ->  memória da tela  ->  o que você vê
            (buffer)             (framebuffer)
            DMA da câmera        DMA do PPA
```

• A câmera MIPI escreve a foto numa área de memória. Isso já é DMA. A função `captureFrame()` pega esse buffer pronto.

• O PPA lê essa foto e escreve o pedaço certo direto na memória da tela DSI. A função que faz isso se chama `blitCameraDmaCropped()`.

• O painel DSI lê essa mesma memória e acende os pontos. Não existe uma terceira cópia para "enviar" a imagem.

Estes exemplos usam tela DSI: Guition 7, Guition 4, Waveshare 10, Functional EV Board, ESP32-P4-EYE e os dois dashboards. Neles o destino do DMA é o framebuffer do `Arduino_DSI_Display`.

O `loop()` do sketch não desenha a câmera. Ele só espera. A tarefa da câmera, criada por `createTask()`, repete o caminho acima.

4. O que o exemplo pede

O trecho abaixo é o `setup` de `examples/generic/ESP32P4/cameras/guition_7in_yellow/guition_7in_yellow.ino`. Os comentários dizem a parte que interessa ao DMA.

```
void setup() {
    Serial.begin(115200);

    // Guarda rotação e espelho. O PPA aplica isso enquanto carrega a foto.
    myDisplay.setRotationCamera(rotationScreen, CAM_MIRROR_X, CAM_MIRROR_Y);

    // Abre o sensor. 1920x1080 é o tamanho da foto na memória, não o tamanho na tela.
    if (myDisplay.startCamera(CAM_I2C_PORT, CAM_SCL, CAM_SDA, CAM_FREQUENCY_SCCB,
                              CAM_RST, CAM_INT, CAM_WIDTH, CAM_HEIGHT)) {
        // Contrata o PPA. A partir daqui a foto pode ir por DMA até a tela.
        myDisplay.getCamera().startPpa();
    } else {
        Serial.println("Camera not started");
    }

    // Sobe o painel DSI. tft->begin() cria o framebuffer, a memória que a tela lê.
    bus = new Arduino_ESP32DSIPanel(/* tempos do JD9165 */);
    tft = new Arduino_DSI_Display(1024, 600, bus, rotationScreen, true, DISP_RST,
                                  jd9165_init_operations,
                                  sizeof(jd9165_init_operations) / sizeof(lcd_init_cmd_t));
    tft->begin();

    // Entrega essa tela à biblioteca. O DMA da câmera passa a escrever nesse framebuffer.
    myDisplay.setDrawObject(tft);

    loadWidgets();                  // Define o retângulo onde a foto vai aparecer.
    myDisplay.loadScreen(screen0);  // Desenha o fundo uma vez.
    myDisplay.createTask(false, 3); // Cria a tarefa que repete a captura e o DMA.
}
```

O retângulo fica em `loadWidgets()`. A câmera continua gravando 1920x1080. O widget só diz o buraco na tela.

```
void loadWidgets() {
    CameraMipiConfig configCameraMipi0 = {
        .width = 232,          // Largura do retângulo na tela, em pontos.
        .height = 184,         // Altura do retângulo na tela.
        .showZoomLevel = false,
        .showFps = false,
        .fps = 5               // A miniatura troca de foto 5 vezes por segundo.
    };

    cameraMipi.setup(configCameraMipi0);
    myDisplay.setCameraMipi(arrayCameraMipi, qtdCameraMipi);
}
```

O `loop()` fica vazio de propósito. Desenhar a câmera aqui, ponto por ponto, seria o caminho sem DMA.

```
void loop() {
    delay(100); // A tarefa da câmera trabalha sozinha.
}
```

Os outros exemplos da pasta repetem esse pedido. Mudam o painel, o sensor e o tamanho do retângulo. O DMA é o mesmo: `startCamera`, `startPpa`, `setDrawObject` e `createTask`.

5. Quem repete isso sozinho

`createTask()` sobe uma tarefa chamada de câmera quando `startCamera()` deu certo. Cada volta faz quatro coisas.

```
void DisplayWX::cameraTask() {
    if (!m_camera.captureFrame()) {
        return; // A câmera ainda não tem uma foto nova.
    }

    ESPVideoBufferClass *buffer = m_camera.buffer(); // Ponteiro da foto. Não é uma cópia.

    // O PPA escreve no framebuffer. O semáforo impede outro desenho ao mesmo tempo.
    if (xSemaphoreTake(m_transactionSemaphore, 0) == pdTRUE) {
        updateCameraMipi(buffer);
        xSemaphoreGive(m_transactionSemaphore);
    }

    m_camera.releaseFrame(); // Devolve o buffer para a câmera gravar a próxima foto.
}
```

`updateCameraMipi()` olha o widget e chama `blitCameraDmaCropped()`. Essa função não tem um `for` nos pontos. Ela monta um pedido e entrega ao PPA.

O pedido real, enxuto, é este:

```
bool DisplayWX::blitCameraDmaCropped(...) {
    uint16_t *fb = m_cameraDsiDisplay->getFramebuffer(); // Memória que a tela já está lendo.

    ppa_srm_oper_config_t op = {};
    op.in.buffer = src;     // Foto da câmera.
    op.in.pic_w = cam_w;    // 1920, por exemplo.
    op.in.pic_h = cam_h;    // 1080, por exemplo.
    op.in.block_offset_x = crop_x; // Onde começa o pedaço usado.
    op.in.block_offset_y = crop_y;
    op.in.block_w = crop_w;
    op.in.block_h = crop_h;

    op.out.buffer = fb;     // Destino: a própria memória da tela.
    op.out.block_offset_x = out_x; // Canto do retângulo dentro dessa memória.
    op.out.block_offset_y = out_y;
    op.scale_x = scale;     // Quanto diminuir ou aumentar.
    op.scale_y = scale;
    op.rotation_angle = cameraPpaAngle(m_cameraRotation);
    op.mode = PPA_TRANS_MODE_BLOCKING; // Espera o DMA terminar.

    ppa_do_scale_rotate_mirror(ppa_srm, &op); // O hardware carrega os pontos.
}
```

`PPA_TRANS_MODE_BLOCKING` quer dizer: espera essa carga acabar antes de devolver o buffer à câmera. Assim a próxima foto não apaga a atual no meio do transporte.

Na miniatura, a tarefa espera o tempo do `fps`. Com `fps = 5`, a espera é 200 ms. Na tela inteira a espera cai para 1 ms, para a imagem acompanhar a câmera.

6. A foto grande vira o retângulo da tela

A foto e o retângulo quase nunca têm a mesma proporção. A biblioteca mede um recorte.

• O recorte fica em volta do centro da foto.

• Zoom 1 mostra o maior pedaço que cabe no retângulo.

• Zoom 2 ou 4 mostram um pedaço menor, e o PPA amplia esse pedaço até encher o retângulo.

• Rotação e espelho, gravados em `setRotationCamera()`, entram no mesmo transporte. Não existe uma segunda passada.

Dois toques no retângulo pedem tela inteira. Aí o destino do DMA deixa de ser o retângulo pequeno e passa a ser a tela toda. Outro toque volta ao retângulo. O sensor continua na mesma resolução. Quem muda é o recorte e o lugar de escrita.

7. A cópia que o processador faz

A imagem ao vivo não passa por `memcpy`. O processador só copia uma foto quando alguém abre o ajuste da câmera, com `openConfig()`.

Essa cópia congela um quadro. As setas e o zoom mexem nesse quadro parado. A câmera pode continuar capturando, mas o ajuste não usa os quadros novos. Ao confirmar ou cancelar, a tarefa volta a escrever ao vivo com o DMA.

Se a serial mostrar `PPA DMA client ready`, o funcionário foi contratado. Se mostrar `failed to start PPA DMA client` ou `Camera not started`, o caminho direto não existe e o retângulo fica sem imagem.